

SubterraView



Group 4

Ashton Edakkunnathu, Manan Patel, Jay Patel, Lena Tran

CS 440

University of Illinois Chicago

Spring 2026

Table of Contents

I	Project Description	8
1	Project Overview	8
2	The Purpose of the Project	8
2a	The User Business or Background of the Project Effort	8
2b	Goals of the Project	8
2c	Measurement	9
3	The Scope of the Work	9
3a	The Current Situation	9
3b	The Context of the Work	9
3c	Work Partitioning	10
3d	Competing Products	11
4	The Scope of the Product	11
4a	Scenario Diagram(s)	12
4b	Product Scenario List	12
4c	Individual Product Scenarios	13
5	Stakeholders	14
5a	The Client	14
5b	The Customer	14
5c	Hands-On Users of the Product	14
5d	Maintenance Users and Service Technicians	15
5e	Other Stakeholders	16
5f	User Participation	16
5g	Priorities Assigned to Users	17
6	Mandated Constraints	17
6a	Solution Constraints	17
6b	Implementation Environment of the Current System	18
6c	Partner or Collaborative Applications	18
6d	Off-the-Shelf Software	19
6e	Anticipated Workplace Environment	19
6f	Schedule Constraints	20
6g	Budget Constraints	20
7	Naming Conventions and Definitions.	20
7a	Definitions of Key Terms	20
7b	UML and Other Notation Used in This Document	21
7c	Data Dictionary for Any Included Models	22
8	Relevant Facts and Assumptions	24
8a	Facts	24
8b	Assumptions	25
II	Requirements	26
9	Product Use Cases	26
9a	Use Case Diagrams	26
9b	Product Use Case List	26

	9c	Individual Product Use Cases	29
10		Functional Requirements	41
11		Data Requirements	43
12		Performance Requirements	45
	12a	Speed and Latency Requirements	45
	12b	Precision or Accuracy Requirements	45
	12c	Capacity Requirements	46
13		Dependability Requirements	46
	13a	Reliability Requirements	46
	13b	Availability Requirements	47
	13c	Robustness or Fault-Tolerance Requirements	47
	13d	Safety-Critical Requirements	47
14		Maintainability and Supportability Requirements	48
	14a	Maintenance Requirements	48
	14b	Supportability Requirements	48
	14c	Adaptability Requirements	49
	14d	Scalability or Extensibility Requirements	49
	14e	Longevity Requirements	49
15		Security Requirements	50
	15a	Access Requirements	50
	15b	Integrity Requirements	50
	15c	Privacy Requirements	50
	15d	Audit Requirements	50
	15e	Immunity Requirements	50
16		Usability and Humanity Requirements	51
	16a	Ease of Use Requirements	51
	16b	Personalization and Internationalization Requirements	52
	16c	Learning Requirements	53
	16d	Understandability and Politeness Requirements	53
	16e	Accessibility Requirements	54
	16f	User Documentation Requirements	54
	16g	Training Requirements	54
17		Look and Feel Requirements	54
	17a	Appearance Requirements	54
	17b	Style Requirements	55
18		Operational and Environmental Requirements	55
	18a	Expected Physical Environment	55
	18b	Requirements for Interfacing with Adjacent Systems	56
	18c	Productization Requirements	56
	18d	Release Requirements	56
19		Cultural and Political Requirements	56
	19a	Cultural Requirements	56
	19b	Political Requirements	57

20	Legal Requirements	57
20a	Compliance Requirements	57
20b	Standards Requirements	57
21	Requirements Acceptance Tests	58
21a	Requirements – Test Correspondence Summary	58
21b	Acceptance Test Descriptions	58
III	Design.....	63
22	Design Goals	63
23	Current System Design.....	65
24	Proposed System Design	65
24a	Initial System Analysis and Class Identification.....	65
24b	Dynamic Modelling of Use-Cases	66
24c	Proposed System Architecture	69
24d	Initial Subsystem Decomposition.....	70
25	Additional Design Considerations.....	71
25a	Hardware / Software Mapping	71
25b	Persistent Data Management	72
25c	Access Control and Security	72
25d	Global Software Control	72
25e	Boundary Conditions.....	73
25f	User Interface	73
25g	Application of Design Patterns	75
26	Final System Design.....	76
27	Object Design	78
27a	Packages	78
27b	User Subsystem.....	78
27c	Application Subsystem.....	80
27d	Data Processing Subsystem.....	83
27e	Field Subsystem	85
27f	Data Storage Subsystem.....	87
27g	Security Subsystem	88
27h	Communication Subsystem.....	90
IV	Project Issues	91
28	Open Issues.....	91
28a	Access to Accurate Utility Data	91
28b	Data Standardization Across Sources	91
28c	AR Inaccuracies	91
28d	811 Integration	91
28e	Legal Liability	91
28f	Crowdsourcing Validation	91
29	Off-the-Shelf Solutions	92
29a	Ready-Made Products	92
29b	Reusable Components	92

	29c	Products That Can Be Copied	92
30		New Problems	92
	30a	Effects on the Current Environment.....	92
	30b	Effects on the Installed Systems.....	93
	30c	Potential User Problems	93
	30d	Limitations in the Anticipated Implementation Environment That May Inhibit the New Product	93
	30e	Follow-Up Problems	93
31		Migration to the New Product	93
32		Risks	93
33		Costs	94
	33a	Development Costs	94
	33b	Hardware Costs	94
	33c	Testing & Development	94
	33d	Total Estimated Initial Monetary Cost	94
	33e	Data Acquisition.....	94
	33f	Maintenance & Updates	94
	33g	Scalability	94
	33h	Total Man-Hours	94
	33i	Development Time	94
	33j	Ongoing Annual Costs, Post-Deployment	94
34		Waiting Room	94
35		Ideas for Solutions	95
36		Project Retrospective	95
V		Glossary	95
VI		References / Bibliography	99
VII		Index	100

List of Figures

Figure 1 – Interaction Diagram.....	9
Figure 2 - Use Case/Scenario Diagram.....	11
Figure 3 - Product Use Case Diagram	25
Figure 4 - Initial Class Diagram.....	64
Figure 5 - Sequence Diagram UC-1.....	65
Figure 6 - Sequence Diagram UC-3.....	65
Figure 7 - Sequence Diagram UC-4.....	66
Figure 8 - Sequence Diagram UC-7.....	67
Figure 9 - Sequence Diagram UC-8.....	68
Figure 10 - Initial Subsystem Package & Class Diagram.....	69
Figure 11 - Deployment Diagram	70
Figure 12 - Final Package Diagram	75
Figure 13 - Final Class Diagram.....	76

List of Tables

Table 1 - Business Event List	11
Table 2 - Product Scenario List	13
Table 3 - Data Dictionary	24
Table 4 - Product Use Case List	29
Table 5 - Requirements - Acceptance Tests Correspondence	58
Table 6 - Risks	94

I Project Description

1 Project Overview

SubterraView is a system that employs Augmented Reality/Artificial Intelligence (AR/AI) to map and visualize underground infrastructure in real time. The system creates a continuously updated map of subterranean infrastructure such as plumbing, telephone cables, and power lines data from compiling public utility records, city databases, and crowdsourced reports. An AI engine cross-references data with each other to validate the information, ranking specifics via reliability scores to resolve information conflicts and retain accuracy. The map is generated through an AR system that allows users to visualize infrastructure directly on their smartphones. The system is meant to provide service to construction workers, utility technicians, and emergency responders who currently rely on the “811 call-before-you-dig” process, a process that introduces significant project delays and opens the possibility of infrastructure damage from human error. SubterraView incorporates a hazard system that provides real-time alerts to workers and systems of dangerous conditions such as gas leaks and structural instabilities, things that are picked up from our portable sensing technology to detect dangers that may go unnoticed by human workers. A crowdsourcing platform allows field workers to report discrepancies, new utilities, and infrastructure conditions—things that could get lost when it comes to human filing—which the AI model uses to continuously update the map.

2 The Purpose of the Project

2a The User Business or Background of the Project Effort

The underground utility and infrastructure management industry supports the installation, maintenance, and repair of underground infrastructure including water and sewage pipelines, cables, and electrical conduits across cities. Currently, contractors must contact 811, a national call before you dig service, to have utility lines marked before any excavation can begin, a process that is time consuming and heavily reliant on outdated records. According to the Common Ground Alliance, nearly 197,000 unique underground utility damage incidents were recorded in 2024 alone [1]. SubterraView is intended to give construction workers, utility technicians, and emergency responders a real time, AI validated map of underground infrastructure accessible directly from their smartphones, eliminating the need to rely solely on the 811 processes.

2b Goals of the Project

The main goal of this project is to develop a reliable and accessible underground infrastructure AI mapping system that enables real time visualization that’s cross-referenced with current validated records, and hazard monitoring for workers operating in subsurface environments. Some of the objectives are to streamline the current notification workflow, minimize accidental utility strikes and the financial damages they cause, provide real time hazard alerts for dangerous conditions (such as gas leaks and structural instabilities), and empower field workers to contribute and access accurate infrastructure data directly from their smartphones.

2c Measurement

- 30 percent or greater reduction in accidental utility strike incidents among active SubterraView users compared to the national average
- Reducing pre-excavation preparation time from the current construction and excavation industry average of several days to 24 hours or less
- AI map accuracy rate reaches a minimum of 90 percent when validated against verified and up-to-date utility records
- A reduction in worker injury rates in subsurface environments where the hazard monitoring system is deployed by determining 85% of hazard alerts must be acted on within 5 minutes of being issued

3 The Scope of the Work

The business environment for SubterraView is municipal underground infrastructure management and safety coordination. The work addressed by this project is the digital detection, validation, visualization, and hazard monitoring of underground infrastructure data to support construction planning and emergency response.

3a The Current Situation

Currently, underground infrastructure management relies on a combination of 811 notification systems, fragmented city and utility databases, surface paint marking, and manual field verification. Before excavation, construction companies must submit requests to 811 services, which notify relevant utility providers. Utility technicians then visit the site to physically mark underground utilities. Not to mention, infrastructure data is often stored across multiple systems maintained by different utility companies which can cause discrepancies in format and inconsistent update frequencies. Emergency responders typically rely on existing city maps or utility contacts and hazard detection systems such as gas sensors, operate independently and aren't always on the mapping platforms. Overall, the process is time consuming and inconsistent. Delays in approval, outdated records, and inconsistent data increase the risk of accidental infrastructure damage and worker injuries.

3b The Context of the Work

The work context diagram below defines the boundary of underground infrastructure management work that SubterraView supports. The central work area encompasses the digital detection, validation, visualization, and hazard monitoring of underground infrastructure data. External entities include 811 services, utility companies, city infrastructure databases, construction contractors, utility technicians, emergency response teams, and hazard sensor networks. Each interface represents a critical data exchange required to support construction planning, excavation safety, and emergency response coordination. The diagram shows both input flows such as infrastructure records, excavation requests, and hazard alerts, and output flows such as infrastructure map overlays, priority emergency maps, and safety notifications. This context establishes the scope of work analysis required to ensure SubterraView integrates seamlessly into the existing underground infrastructure management ecosystem.

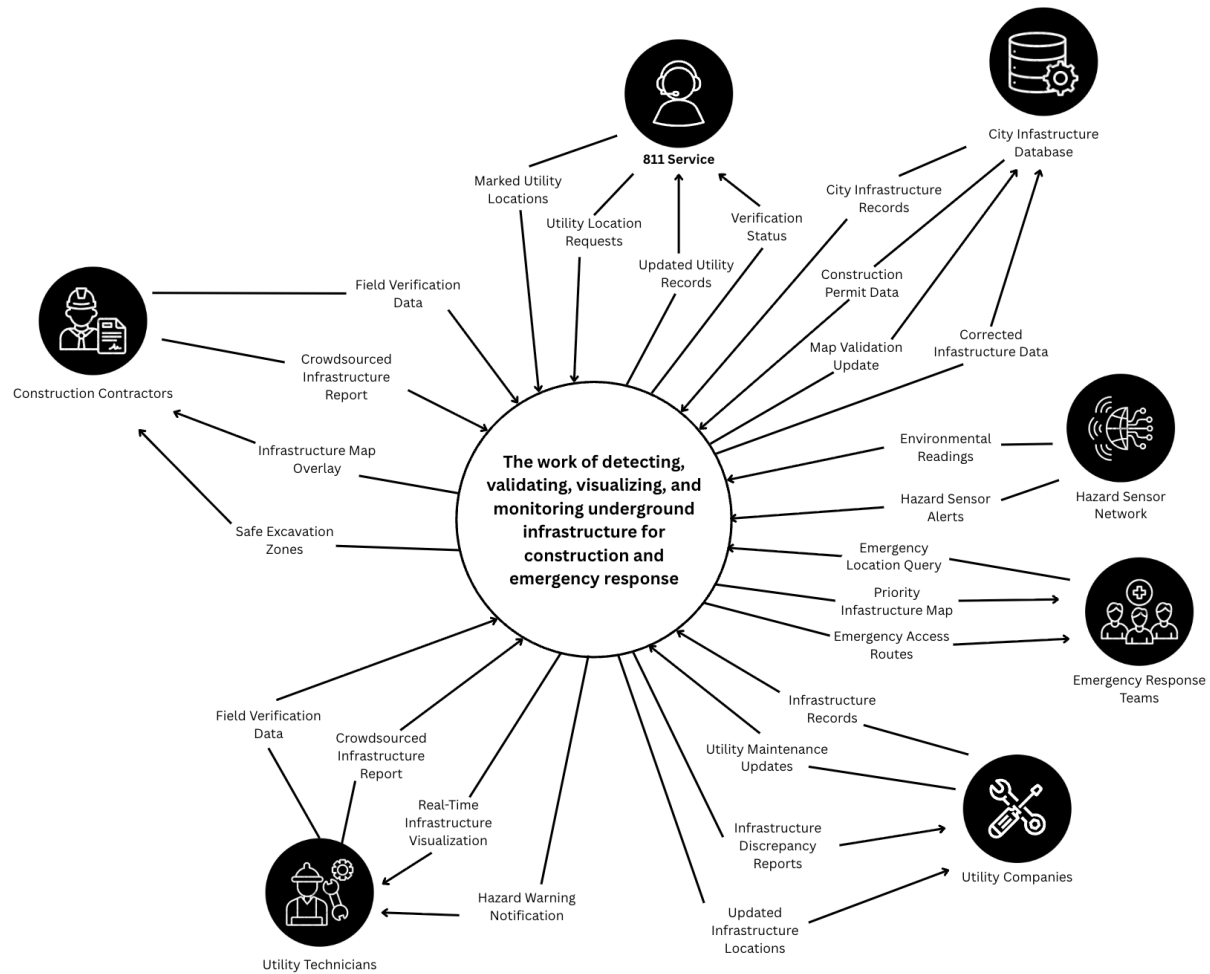


Figure 1 – Interaction Diagram

3c Work Partitioning

Event Name	Input and output	Summary
Construction Request Initiated	Excavation Location Request (in), Project Site Coordinates (in), Infrastructure Map Overlay (out), Safe Excavation Zones (out)	When a construction project begins, the system retrieves and displays underground infrastructure data for the selected area.
811 Service Request Received	Utility Location Requests (in), Marked Utility Locations (in), Verification Status (out), Updated Utility Records (out)	When an 811 request is received, the system processes utility location data and returns verification status and updated records.

City Infrastructure Data Exchange	City Infrastructure Records (input), Construction Permit Data (input), Map Validation Update (output), Corrected Infrastructure Data (output)	When city infrastructure or permit data is received, the system validates and corrects the infrastructure map accordingly.
Utility Data Updated	Infrastructure Records (in), Utility Maintenance Updates (in), Updated Infrastructure Locations (out), Infrastructure Discrepancy Reports (out)	When new or modified utility records are received, the system validates and integrates the data into the map and reports any discrepancies.
Hazard Detected	Hazard Sensor Alert (in), Environmental Readings (in), Hazard Warning Notification (out)	When abnormal underground conditions are detected, the system alerts workers and relevant authorities.
Crowdsourced Report Submitted	Crowdsourced Infrastructure Report (in), Field Verification Data (in), Map Validation Update (out), Corrected Infrastructure Data (out)	When a worker reports a discrepancy or new infrastructure, the system verifies and updates the model.
Emergency Response Initiated	Emergency location query (in), Priority Infrastructure Map (out), Emergency Access Routes (out)	When emergency responders request information, the system provides a focused underground layout of the affected area.

Table 1 - Business Event List

3d Competing Products

Several existing systems address underground infrastructure mapping and safety coordination, but only partially. The most widely used alternative is the 811 service, which notifies utility providers before excavation begins. While effective for basic coordination, this process is manual, time consuming, and does not provide real-time visualization. There are also other products such as Geographic Information System (GIS) and Ground-Penetrating Radar (GPR), but they are typically static, fragmented, and don't integrate data into a centralized infrastructure model. Although these solutions provide partial functionality, none combine AI-based validation, AR visualization, real-time updates, and integrated hazard detection into a unified system. SubterraView aims to address these gaps by offering a comprehensive, underground infrastructure platform.

4 The Scope of the Product

SubterraView handles the digital detection, validation, visualization, and hazard monitoring of underground infrastructure data. The product does not handle physical excavation, regulatory enforcement, or long-term infrastructure maintenance. These remain the responsibility of external organizations. The scenarios below demonstrate

how construction workers, utility technicians, emergency responders, and system administrators would interact with SubterraView in real-world situations.

4a Scenario Diagram(s)

The scenario diagram below illustrates the scope of the SubterraView product and the interactions between external actors and system functions. The diagram shows three types of users on the left: construction workers, utility technicians, and system administrators. On the right are the external systems that provide data to or receive data from SubterraView: emergency responders, city infrastructure database, utility database, and the hazard sensor network. The central box contains the core functions of the product including viewing underground infrastructure, validating utility records, receiving hazard alerts, accessing emergency mode, reviewing data accuracy reports, and submitting crowdsourced infrastructure reports. Each arrow represents a specific interaction between an actor and a system function, with the complete set of interactions mapping to the individual product scenarios described in section 4c.

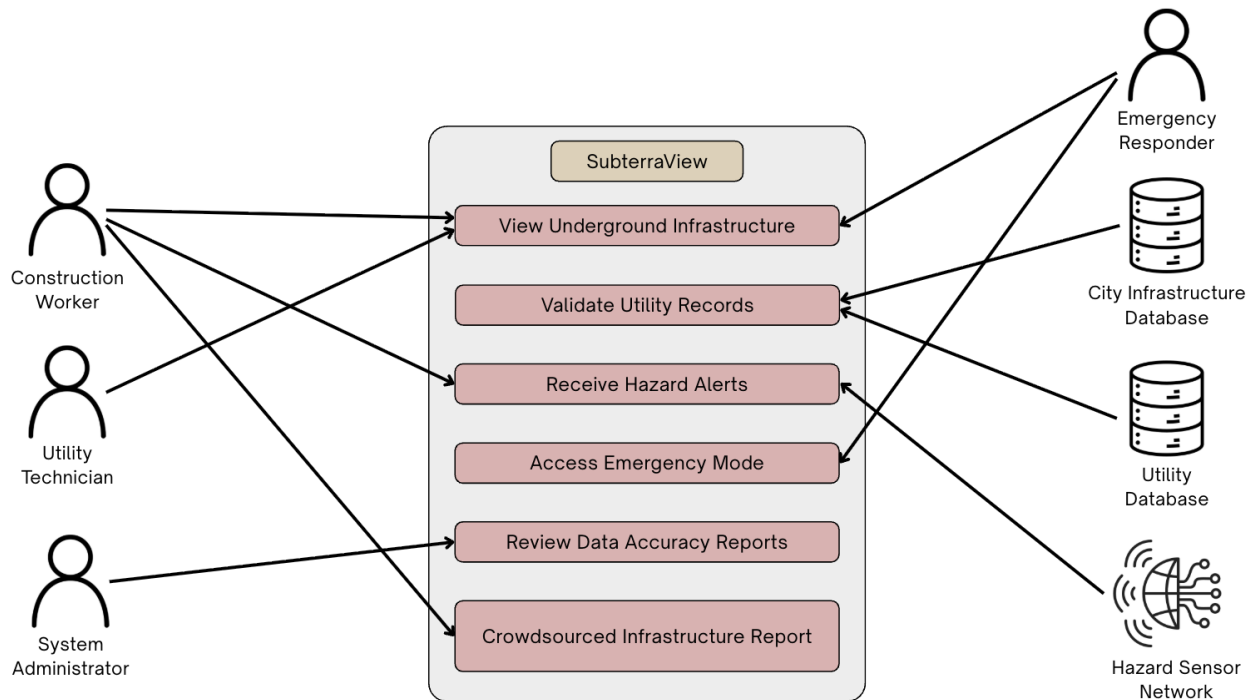


Figure 2 - Use Case/Scenario Diagram

4b Product Scenario List

Scenario Name	External Actors Involved
---------------	--------------------------

1. AR Underground Visualization	Construction Worker
2. Utility Data Update and Validation	Utility Database, City Infrastructure Database
3. Crowdsourced Infrastructure Report	Utility Technician
4. Hazard Detection and Alert Notification	Hazard Sensor Network, Construction Worker, Emergency Responder
5. Emergency Infrastructure Access	Emergency Responder
6. Periodic Data Accuracy Review	System Administrator

Table 2 - Product Scenario List

4c Individual Product Scenarios

Scenario 1: AR Underground Visualization

A construction worker arrives at a new job site to begin excavation. Instead of waiting for surface painting markings or contacting multiple utility providers, the worker opens the SubterraView application on their smartphone. After selecting the project location, the app overlays underground infrastructure directly onto the camera view, showing water lines, gas pipes, electrical conduits, and telecommunications cables beneath the surface. Each utility line is color-coded and labeled for clarity. The worker adjusts the viewing angle and zooms level to better understand the layout and identifies a safe excavation path. This allows the team to begin work confidently and efficiently without unnecessary delays.

Scenario 2: Utility Data Update and Validation

A utility company completes an upgrade to a section of underground electrical wiring and updates their records in the utility database. The updated infrastructure records are automatically submitted to SubterraView. The system integrates the new data into its centralized model and cross-references it with existing records from the city infrastructure database. If conflicting information is detected, the system flags the discrepancy for review and assigns a confidence rating to the updated section. Once validated, the infrastructure map is updated so that future construction crews and responders have access to the most accurate information available.

Scenario 3: Crowdsourced Infrastructure Report

While working underground, a utility technician discovers a cable that is not shown on the digital map. Using the SubterraView app, the technician submits a report including the location, description, and supporting images. The system records the submission and compares it with existing infrastructure data and previous reports. After review, the infrastructure model is updated, and the confidence rating for that area is adjusted accordingly. This ensures that the system continuously improves and reflects real-world conditions.

Scenario 4: Hazard Detection and Alert Notification

During excavation, a connected underground gas sensor detects abnormal methane levels. The hazard monitoring system immediately sends a warning notification to the construction worker through the SubterraView application. The AR interface highlights the affected underground area in red. Workers evacuate the area immediately and emergency responders are notified of the hazard location to prevent potential injuries or explosions.

Scenario 5: Emergency Infrastructure Access

An underground structural collapse occurs during a construction project. Emergency responders arrive at the scene and access SubterraView in emergency mode. By entering the location, responders instantly view a prioritized underground map showing utility lines, confined spaces, and possible hazards. This allows the response team to determine the safest entry points and identify potential danger zones before beginning rescue operations.

Scenario 6: Periodic Data Accuracy Review

At scheduled intervals, a system administrator initiates a data integrity review of stored infrastructure records. The system analyzes the map and flags sections with outdated or low confidence data for further inspection. The administrator receives a detailed report identifying areas that may require updated verification or additional field data. Using this report, the administrator coordinates follow-up actions to ensure the infrastructure map remains accurate and reliable over time.

5 Stakeholders

5a The Client

The primary client for SubterraView is municipal public works departments. These are government agencies responsible for managing and maintaining underground infrastructure including water, sewer, and electrical systems across cities. Municipal public works departments have a direct financial and operational interest in reducing utility damage incidents and improving excavation safety coordination within their jurisdictions.

5b The Customer

The primary customers for SubterraView are construction companies and utility companies. Construction companies represent the largest end-user segment in the underground utility mapping market, driven by the need for accurate subsurface data before excavation begins. Utility companies are also key customers, as they rely on accurate infrastructure mapping to manage, maintain, and update their underground networks. Emergency response agencies represent a secondary customer group that would adopt SubterraView for hazard coordination purposes.

5c Hands-On Users of the Product

1. Construction Worker:

- (a) User role: Uses SubterraView in the field to view underground infrastructure before and during excavation, reducing the risk of accidental utility strikes.
- (b) Subject matter experience: Journeyman to Master, as familiarity with underground infrastructure on the field is expected.
- (c) Technological experience: Novice to Journeyman, based on general comfort with mobile technology.

- (d) Other user characteristics: Field based workers who need quick and reliable access to data while on site. May work in loud or physically demanding environments where ease of use is important.
- 2. Utility Technician
 - (a) User role: Uses SubterraView to verify infrastructure records, submit crowdsourced reports, and flag discrepancies found during field work.
 - (b) Subject matter experience: Master, as utility technicians are expected to have strong knowledge of underground systems and infrastructure.
 - (c) Technological experience: Journeyman, as comfort with digital reporting tools is expected in this role.
 - (d) Other user characteristics: Often works independently in the field and relies on accurate data to complete maintenance and inspection tasks safely.
- 3. Emergency Responders:
 - (a) User role: Uses SubterraView in emergency mode to access real time underground infrastructure maps during active incidents such as structural collapses or gas leaks.
 - (b) Subject matter experience: Master, as emergency responders are expected to operate quickly and accurately under pressure.
 - (c) Technological experience: Journeyman to Master, as above average comfort with technology is necessary when responding to time sensitive emergencies.
 - (d) Other user characteristics: Response needs will vary depending on the type of underground emergency. Fast load times and a clear interface are critical for this user group.
- 4. Household Individuals:
 - (a) User role: Able to use SubterraView for personal projects allowing for better planning.
 - (b) Subject matter experience: Novice to Journeyman, as some basic knowledge of underground aspects may be required.
 - (c) Technological experience: Novice to Journeyman, comfortable with mobile apps and understanding basic visualizations.
 - (d) Other user characteristics: Use cases will vary from casual app usage to hands-on personal projects. Basic knowledge regarding underground aspects should be known.

5d Maintenance Users and Service Technicians

- 1. Software Engineers and AI Engineers:
 - (a) Responsible for deploying updates, resolving software bugs, and improving core features such as the AR visualization engine and AI validation model to keep SubterraView functional and accurate over time.
- 2. IT Support Technicians:
 - (a) Description: Responsible for resolving hardware compatibility issues, managing device support requests, and assisting users who experience technical difficulties with the application.
- 3. Database Administrators:
 - (a) Responsible for maintaining the integrity and security of stored infrastructure records, managing data pipelines from external sources

such as the City Infrastructure Database and Utility Database, and ensuring data is properly formatted and accessible.

5e Other Stakeholders

1. Testers:
 - (a) Knowledge needed: Software testing experience to identify bugs and performance issues, as well as working knowledge of underground infrastructure to validate that the app functions accurately in real world conditions.
 - (b) Degree of involvement: High, since testers will be involved throughout the development cycle, providing structured feedback and verifying that issues are resolved before deployment.
 - (c) Degree of influence: High, since their findings directly affect whether SubterraView is approved for release.
2. Legal Experts
 - (a) Knowledge needed: Knowledge of federal and state regulations governing excavation safety, including 811 compliance requirements, to ensure SubterraView operates within all applicable legal boundaries.
 - (b) Degree of involvement: Low, since legal experts will be consulted during the early stages of development and when significant changes to the product are made.
 - (c) Degree of influence: High, since SubterraView cannot be deployed without confirming that it complies with all relevant regulations.
 - (d) Conflict resolution: If legal requirements conflict with planned features, legal compliance takes precedence and the feature will be modified or removed accordingly.
3. Sponsors
 - (a) Knowledge needed: General understanding of the underground infrastructure management industry and the value SubterraView provides to construction and utility operations.
 - (b) Degree of involvement: Low, since sponsors are primarily involved during funding decisions and major development milestones.
 - (c) Degree of influence: High, since sponsor funding directly affects the pace and scope of development.
 - (d) Conflict resolution: If sponsor expectations conflict with technical or legal requirements, the development team will communicate constraints clearly and propose adjusted timelines or feature scopes.

5f User Participation

1. Construction Worker:
 - (a) Description: Construction workers will contribute field level feedback on the accuracy of the AR infrastructure overlay and the usability of the interface in active job site conditions. Their input will be essential for validating that the system performs reliably during excavation planning.
 - (b) Time contribution: Involved during initial testing phases and on an ongoing basis after deployment through the crowdsourced reporting feature.
2. Utility Technician

- (a) Description: Utility technicians will provide subject matter expertise on infrastructure data accuracy and help validate that crowdsourced reporting functions correctly in the field. Their feedback will be used to refine the confidence rating system.
 - (b) Time contribution: Involved during testing phases, particularly for scenarios related to data validation and discrepancy reporting.
- 3. Emergency Responders:
 - (a) Description: Utility technicians will provide subject matter expertise on infrastructure data accuracy and help validate that crowdsourced reporting functions correctly in the field. Their feedback will be used to refine the confidence rating system.
 - (b) Time contribution: Involved during testing phases, particularly for scenarios related to data validation and discrepancy reporting.

5g Priorities Assigned to Users

- 1. Key users: Construction Worker and Utility Technician - 70%
 - (a) Construction workers and utility technicians are the primary users of SubterraView and represent the core use cases the product is designed to support. Their day-to-day reliance on accurate infrastructure data means their requirements take the highest priority in design and development decisions. Feedback from these users has the greatest influence on product iterations.
- 2. Secondary users: Emergency Responders - 20%
 - (a) Emergency responders represent an important but situational user group. While their use of SubterraView is less frequent than that of construction workers and utility technicians, the consequences of system failure during an emergency response are significant. Their requirements are given strong consideration but take lower priority than those of key users when conflicts arise.
- 3. Unimportant users: System Administrator- 10%
 - (a) System administrators interact with SubterraView primarily through backend data management tools rather than the core application interface. Their requirements are given the lowest design priority, though their role in maintaining data accuracy is still essential to overall system reliability.

6 Mandated Constraints

6a Solution Constraints

- 1. 811 Services:
 - (a) Description: 811 is a federally mandated call-before-you-dig service that is contacted before digging takes place. SubterraView will abide by this mandate and incorporate it into the application. SubterraView shall incorporate reminders and guidance for users to contact 811 services before any excavation begins, in accordance with federal mandate.
 - (b) Rationale: SubterraView is designed to supplement the 811 processes, not replace it. All excavation activity must comply with federal call before you dig requirements regardless of what the app displays.
 - (c) Fit Criterion: SubterraView shall display a prompt directing Construction Workers to contact 811 before excavation begins on any new project site.

2. Data Accuracy and Validation:
 - (a) Description: SubterraView shall use an AI engine to cross reference data from multiple sources including public utility records, city infrastructure databases, and crowdsourced reports. The system shall assign a confidence rating to all infrastructure data and shall not present assumed or unverified information as confirmed.
 - (b) Rationale: The reliability of the AR infrastructure overlay depends entirely on the accuracy of the underlying data. Displaying inaccurate infrastructure locations could result in utility strikes or worker injuries.
 - (c) Fit Criterion: The AI engine shall flag any data that falls below the minimum confidence threshold and shall clearly indicate to the user when data in a given area is unverified or potentially outdated.
3. Platform and Hardware Compatibility:
 - (a) Description: SubterraView shall be compatible with both iOS and Android smartphones. External hazard detection sensors shall connect to the app via Bluetooth and shall meet minimum weight and size requirements so as not to interfere with standard field equipment.
 - (b) Rationale: Construction Workers and Utility Technicians operate in field environments where a lightweight, smartphone-based solution is the most practical and accessible option. Modern smartphones do not have built in gas or hazard detection capabilities, making minimal external hardware necessary.
 - (c) Fit Criterion: SubterraView shall be approved for deployment on both iOS and Android platforms. External sensors shall weigh no more than 500 grams and shall pair with the app via Bluetooth without requiring additional software installation.

6b Implementation Environment of the Current System

SubterraView will operate primarily in outdoor field environments where Construction Workers, Utility Technicians, and Emergency Responders access the application through iOS and Android smartphones. The app requires an active internet connection to retrieve infrastructure data from the City Infrastructure Database and Utility Database in real time. In areas with limited connectivity, the system shall cache recently accessed infrastructure data locally on the device to maintain basic functionality.

External Bluetooth sensors will be deployed at job sites to detect hazardous underground conditions such as gas leaks and structural instabilities. These sensors will communicate with the SubterraView application wirelessly and trigger hazard alerts through the Hazard Monitoring System. The app will rely on the device camera for AR visualization and GPS for precise location mapping. All data processed by the system will pass through the AI validation engine before being displayed to the user.

6c Partner or Collaborative Applications

1. 811 Service Applications
 - (a) SubterraView must interface with the 811-service notification system to ensure users are directed to complete the required call before you dig process before excavation begins. This integration supports compliance with federal excavation safety requirements and ensures SubterraView operates as a supplement to the existing 811 process rather than a replacement.

2. Hazard Sensor Network
 - (a) SubterraView must communicate with external hazard detection sensors via Bluetooth to receive real time environmental readings. The app must be capable of processing incoming sensor data and triggering hazard alerts through the Hazard Monitoring System within a timeframe that meets OSHA worker protection requirements.
3. GIS Platform
 - (a) SubterraView must integrate with an established geospatial platform to support the mapping and AR visualization functions of the application. This integration allows SubterraView to accurately position underground infrastructure data relative to real world coordinates and ensures that the infrastructure map overlay aligns correctly with the physical environment as viewed through the smartphone camera.

6d Off-the-Shelf Software

1. Cloud Services (Google Cloud Platform or Amazon Web Services) - SubterraView will use an established cloud platform to host the AI validation engine, store infrastructure data, and manage real time data transfers between the app and external databases. These platforms provide the scalability and reliability needed to support multiple simultaneous users across different job sites.
2. AI/ML frameworks (TensorFlow or PyTorch) - SubterraView will use an established machine learning framework to build and deploy the AI engine responsible for cross referencing infrastructure data and assigning confidence ratings. These frameworks provide prebuilt tools for model training and data validation that reduce development time.
3. GIS/GPS Software (Esri ArcGIS or Google Maps Platform) - SubterraView will integrate with an established geospatial platform to support real time location mapping and AR overlay functionality. These platforms provide the coordinate systems and mapping APIs necessary for accurately positioning underground infrastructure data relative to the physical environment.
4. Databases (PostgreSQL with PostGIS extension) - SubterraView will use a relational database system with geospatial support to store and manage infrastructure records, confidence ratings, and crowdsourced reports. PostGIS extends PostgreSQL with geographic data types, making it well suited for storing and querying location-based infrastructure data.
5. AR Development Framework (ARKit (iOS) and ARCore (Android)) - SubterraView will use Apple ARKit and Google ARCore to power the augmented reality visualization features of the application. These are the native AR development frameworks for their respective platforms and are required to ensure stable AR performance across supported devices.

6e Anticipated Workplace Environment

SubterraView will be used primarily in outdoor environments including active construction sites and utility work zones. These environments present several design challenges that the product must account for. Construction sites are typically loud, with heavy machinery operating nearby, so SubterraView will use visual alerts and phone vibration in addition to audio notifications to ensure hazard warnings reach the user regardless of ambient noise levels. Outdoor use also means the display must

remain readable in direct sunlight, requiring high contrast interface design. External hardware sensors must be weather resistant to function reliably in rain or dusty field conditions. Since Construction Workers and Utility Technicians may be wearing gloves or operating with limited hand mobility, the interface must support simple touch interactions and minimize the need for precise input.

6f Schedule Constraints

SubterraView estimates a total development timeline of approximately eighteen months. The first twelve months will cover system design, AI engine development, database integration, and AR feature implementation. The following six months will be dedicated to structured testing, bug fixes, and initial feedback collection from a selected group of Construction Workers and Utility Technicians before the full release on iOS and Android platforms.

Meeting this timeline is critical because the underground utility mapping market is growing rapidly and delays in release could result in competing products capturing the target customer base first. If the 18-month deadline is not met, the development team will prioritize core features including AR visualization, AI data validation, and hazard alerting, and defer lower priority features to a post launch update cycle rather than delay the full release.

6g Budget Constraints

The budget for SubterraView must account for several major cost areas including cloud infrastructure, AI engine development and training, GIS and mapping platform licensing, external sensor hardware development, and mobile application development for both iOS and Android. Given that SubterraView is being developed as a student project, the initial budget is limited, and the development team will prioritize the use of open-source frameworks and free tier cloud services where possible to reduce costs during the development phase.

A realistic minimum budget for a production ready version of SubterraView would need to cover ongoing API licensing fees, cloud hosting costs, and hardware prototyping. If funding falls short of what is required to build and maintain all planned features, the team will focus resources on the core AR visualization and hazard alerting functions, as these represent the highest priority requirements for the primary user groups.

7 Naming Conventions and Definitions.

7a Definitions of Key Terms

Augmented Reality: A technology that overlays digital information and visual elements onto the real-world environment using devices such as smartphones. In SubterraView, augmented reality is used to display underground infrastructure directly over a live camera view.

Artificial Intelligence: A computational system that takes large datasets, detects patterns, and makes decisions or predictions based on the information given. In SubterraView, AI is used to cross-reference underground infrastructure data from multiple sources, assign reliability scores, and update the infrastructure model.

Subterranean Infrastructure: Underground systems that support municipal and utility operations, including water pipelines, sewage systems, gas lines, electrical conduits, and telecommunications cables.

AI Validation Engine: An AI component that verifies infrastructure data by comparing records from utility databases, city infrastructure databases, and crowdsourced reports. It assigns confidence ratings and resolves conflicting data before updating the infrastructure map.

Confidence Rating: A numerical or categorical score generated by the AI system that represents the reliability and verification level of mapped infrastructure data.

Crowdsourcing: The process of collecting information from field workers or system users who submit reports about infrastructure discrepancies, new utilities, or hazard observations through the SubterraView application.

Sensor Units: Hardware devices that detect underground hazards such as gas leaks, structural instability, or abnormal environmental conditions and transmit alerts to the SubterraView system.

Hazard Monitoring System: A system module that receives input from sensor units and analyzes environmental risks to provide real-time safety alerts to workers and emergency responders.

7b UML and Other Notation Used in This Document

This document generally follows the UML 2.0 standard for all diagrams and notations. Any exceptions or custom symbols are noted below.

1. Interaction Diagram (Figure 1)
 - (a) Symbol: Large white circle in the center
 - i. Meaning: Core system functions
 - (b) Symbol: Black circles at the edges
 - i. Meaning: External users, interacting with the system
 - (c) Symbol: Solid black arrows
 - i. Meaning: What the users give to the system and vice versa
2. Use Case Diagram (Figure 2)
 - (a) Symbol: Circle on top of an open semicircle
 - i. Meaning: External user that interacts with the system
 - (b) Symbol: Database cylinder
 - i. Meaning: External data that interacts with the system
 - (c) Symbol: Alarm
 - i. Meaning: Hazard Detection System
 - (d) Symbol: Solid arrow between an external role and use case
 - i. Meaning: Interactions/Initiates use case
3. Class Diagram (Figures 4, 10, 13)
 - (a) Symbol: Plain rectangle
 - i. Meaning: A class in the system
 - (b) Symbol: Hollow triangle arrowhead
 - i. Meaning: Inheritance
 - (c) Symbol: Filled diamond
 - i. Meaning: Composition
 - (d) Symbol: Hollow diamond
 - i. Meaning: Aggregation

- (e) Symbol: Plain solid arrow
 - i. Meaning: Association
- 4. Sequence Diagrams (Figures 5-9)
 - (a) Symbol: Stick figure
 - i. Meaning: A human actor initiating the use case
 - (b) Symbol: Rectangle with underlined name
 - i. Meaning: An object or class participating in the sequence
 - (c) Symbol: Vertical dashed line
 - i. Meaning: Represents the existence of an object over time
 - (d) Symbol: Solid arrow between lifelines
 - i. Meaning: A method call or message sent from one object to another
 - (e) Symbol: Dashed arrow between lifelines
 - i. Meaning: A return message sent back to the calling object
- 5. Deployment Diagram (Figure 11)
 - (a) Symbol: 3D cube shape
 - i. Meaning: A physical hardware device or node
 - (b) Symbol: Rectangle inside a node labeled with <<execution environment>>
 - i. Meaning: The software platform running on that hardware
 - (c) Symbol: Rectangle with component icon labeled with <<component>>
 - i. Meaning: A software subsystem deployed on that node
 - (d) Symbol: Solid line between nodes labeled with <<protocol>>
 - i. Meaning: A communication pathway between hardware nodes
- 6. Package Diagram (Figure 12)
 - (a) Symbol: Rectangle with a small tab in the top left corner
 - i. Meaning: A package or subsystem grouping related classes
 - (b) Symbol: Plain rectangle inside a package
 - i. Meaning: A class belonging to that subsystem
 - (c) Symbol: Dashed arrow between packages
 - i. Meaning: A dependency relationship between two subsystems

7c Data Dictionary for Any Included Models

Flow/Store Name	Type	Source, Destination	Description
Excavation Location Request	Information Flow	SubterraViewApp to ValidationEngine	A request submitted by a field worker containing GPS coordinates and site details, triggering retrieval of underground infrastructure data.
Infrastructure Data Query	Information Flow	ValidationEngine to InfrastructureDatabase	A structured query sent to external utility and municipal databases requesting underground infrastructure records.

Raw Infrastructure Records	Information Flow	InfrastructureDatabase to ValidationEngine	Unverified infrastructure records returned from utility companies and city databases in response to an infrastructure data query.
Validated Infrastructure Model	Information Store	ValidationEngine	The verified and confidence-rated infrastructure map maintained by the system.
Infrastructure Map Overlay	Information Flow	ValidationEngine to AR Overlay	A structured data package containing georeferenced 3D infrastructure models ready for display.
AR Overlay Data Package	Information Flow	AROverlay to SubterraViewApp	A rendered color-coded visualization of underground infrastructure displayed on the user's device screen.
Safe Excavation Zones	Information Flow	ValidationEngine to SubterraViewApp	Identified areas where excavation can safely proceed based on validated infrastructure data.
Crowdsourced Field Report	Information Flow	SubterraViewApp to ValidationEngine	A user-submitted report containing observed infrastructure including location, type, description, and images.
Crowdsourced Report Confirmation	Information Flow	ValidationEngine to SubterraViewApp	A confirmation message sent back to the user after a field report has been successfully received and queued for review.
Hazard Sensor Reading	Information Flow	SensorUnit to HazardMonitoringSystem	Real-time sensor data transmitted from deployed Bluetooth sensor units detecting environmental conditions.
Hazard Warning Notification	Information Flow	HazardMonitoringSystem to SubterraViewApp	An urgent alert generated by the HazardMonitoringSystem when abnormal

			conditions are detected, containing hazard type, severity, and location.
Emergency Location Query	Information Flow	SubterraViewApp to ValidationEngine	A request submitted by an Emergency Responder containing the incident location, triggering retrieval of a priority infrastructure map.
Priority Infrastructure Map	Information Flow	ValidationEngine to SubterraViewApp	A focused underground infrastructure map generated for emergency responders highlighting utility lines, hazard zones, and safe entry points.
User Authentication Request	Information Flow	SubterraViewApp to AuthenticationService	A request containing user credentials and selected role submitted at login for verification.
Session Token	Information Flow	AuthenticationService to SubterraViewApp	A unique token issued after successful authentication, used to maintain the user's access level throughout the session.
Cached Infrastructure Data	Information Store	NetworkCommunicationService	Previously retrieved infrastructure records stored locally on the device for use during low connectivity situations.

Table 3 - Data Dictionary

Infrastructure image data is stored in JPEG or PNG format, telemetry data is stored in CSV format, and AR overlay data is stored in GLTF format.

8 Relevant Facts and Assumptions

8a Facts

Underground infrastructure management currently relies heavily on the 811 call-before-you-dig notification system.

The 811 process takes multiple days and relies on field markings.

Nearly 197,000 underground utility damage incidents were reported in 2024 according to the Common Ground Alliance DIRT Report.

Underground utility damage incidents cost an estimated \$30 billion annually in societal and economic losses.

SubterraView depends on data provided by utility companies and municipal agencies.

Hazard monitoring depends on sensor units detecting gas leaks, structural risks, or environmental abnormalities.

The system supports construction planning, excavation safety, and emergency response coordination.

SubterraView does not perform physical excavation, infrastructure maintenance, or regulatory enforcement.

8b Assumptions

Utility companies and municipal agencies will allow access to infrastructure data.

Users will receive training to operate the SubterraView application.

Reliable internet connectivity will usually be available during system operation.

AI validation will maintain approximately 90% infrastructure data accuracy.

Infrastructure repair and regulatory compliance responsibilities remain with external organizations.

II Requirements

9 Product Use Cases

9a Use Case Diagrams

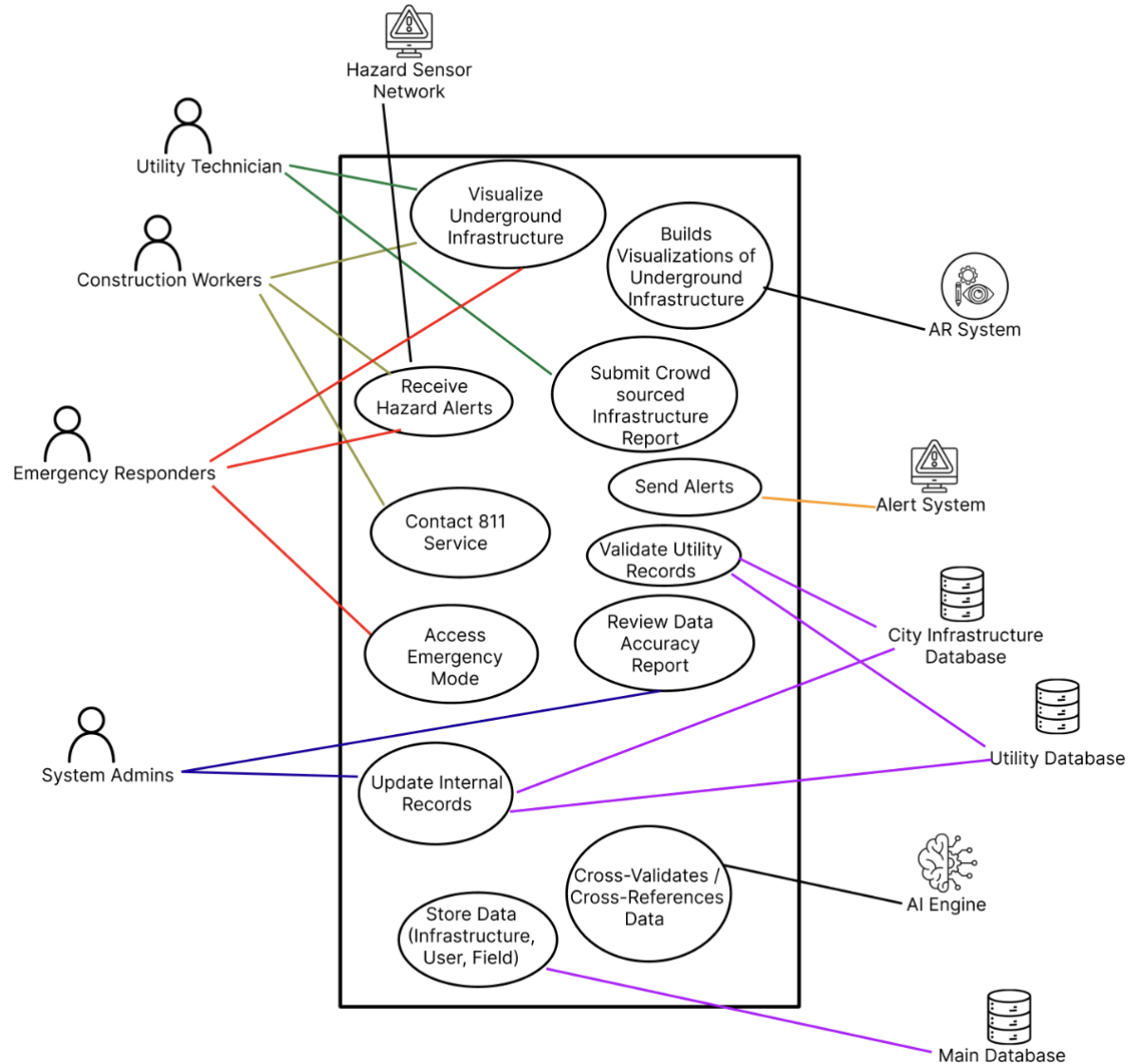


Figure 3 - Product Use Case Diagram

9b Product Use Case List

Use Case ID	Use Case Name	Description	Actors
UC-1	Visualize Underground Infrastructure	User views AR overlay of underground infrastructure for a selected site	Construction Worker, Utility Technician, Emergency Responder

UC-2	Build Visualizations of Underground Infrastructure	AR system builds a color coded underground infrastructure overlay using validated data	AR System
UC-3	Submit Crowdsourced Infrastructure Report	User submits a field report about undocumented or incorrect infrastructure	Utility Technician
UC-4	Receive Hazard Alert	User receives a real time hazard notification triggered by the Hazard Sensor Network	Construction Workers, Emergency Responder, Hazard Sensor Network
UC-5	Send Alerts	Alert system broadcasts hazard notifications to all relevant users in the affected area	Alert System
UC-6	Contact 811 Service	User accesses 811 services through the app before any excavation takes place	Construction Workers
UC-7	Validate Utility Records	System cross references and validates incoming infrastructure data from external sources	Utility Database, City Infrastructure Database
UC-8	Access Emergency Mode	Emergency responder accesses a prioritized underground infrastructure map during an active incident	Emergency Responder
UC-9	Review Data Accuracy Report	System Administrator initiates a periodic review of infrastructure data integrity	System Administrator
UC-10	Update Internal Records	System processes and stores approved infrastructure updates after validation	Utility Database, City Infrastructure Database, System Administrator

UC-11	Store Data (Infrastructure, User, Field)	System stores all infrastructure, user, and field data for access by internal components	Main Database
UC-12	Cross-Validate/ Cross-References Data	AI engine processes and verifies data from multiple sources and assigns confidence ratings	AI Engine

Table 4 - Product Use Case List

9c Individual Product Use Cases

Use case ID: UC-1 Name: Visualize Underground Infrastructure pre-conditions: The SubterraView application is running on a compatible iOS or Android device. The device camera and GPS are enabled. The AI validation engine and AR system are online. post-conditions: The AR system displays a color-coded overlay of underground infrastructure for the selected site on the user's smartphone screen. Initiated by: Construction Worker, Utility Technician, Emergency Responder Triggering Event: Construction Worker opens the SubterraView application and selects or confirms a project site location. Additional Actors: City Infrastructure Database, Utility Database
Sequence of Events: 1. The Construction Worker opens SubterraView and grant's location access. a. The system detects the device location via GPS and displays the confirmed site coordinates. 2. The Construction Worker confirms the site location. a. The system sends the location data to the AI validation engine, which retrieves infrastructure records from the City Infrastructure Database and Utility Database. 3. The AI validation engine retrieves infrastructure records from the City Infrastructure Database and Utility Database. a. The system cross references the retrieved records and assigns confidence ratings to each infrastructure element. 4. The system passes the validated data to the AR system. a. The AR system builds a color-coded underground infrastructure overlay and displays it on the user's screen.

Alternatives: If GPS is unavailable, the Construction Worker may manually enter the site address or coordinates, and the system will retrieve records for the entered location.

Exceptions: If no network connection is available, the system will notify the Construction Worker and display any previously cached infrastructure data for the area if available. If no infrastructure records exist for the selected location, the system will notify the Construction Worker that data is unavailable for that site.

Notes: Additional Actors are not outlined in the diagram directly but instead is referenced here and visualized via. The other use cases where the databases are the initiator!

Use case ID: UC-2
Infrastructure

Name: Build Visualizations of Underground

pre-conditions: The SubterraView application is running. The AR system is online. Infrastructure records are available for the selected site.

post-conditions: The AR system produces a color-coded visualization of the underground infrastructure that is ready for display on the user's screen.

Initiated by: AR System

Triggering Event: User selects or confirms a site location, and the system begins building the visualization.

Additional Actors: City Infrastructure Database, Utility Database

Sequence of Events:

1. The user selects or confirms the site location.
 - a. The system sends the location data to the AR system.
2. The AR system requests the relevant infrastructure data from the City Infrastructure Database.
 - a. The system retrieves and passes the available records to the AR system.
3. The AR system processes the data and assigns confidence ratings to each infrastructure element.
 - a. The system builds a color-coded underground infrastructure visualization.
4. The AR system confirms the visualization is ready for display.

a. The system displays the completed visualization to the user.
Alternatives: N/A Exceptions: If site records are unavailable or limited, the AR system will notify the user that the visualization may be incomplete. If the AR system goes offline, the system will notify the user and fall back to the standard map view. Notes: Additional Actors are not outlined in the diagram directly but instead is referenced here and visualized via. the other use cases where the databases are the initiator!

Use case ID: UC-3 Name: Submit Crowdsourced Infrastructure Report pre-conditions: The SubterraView application is running. The Utility Technician is logged in and has an active network connection. post-conditions: The submitted report is stored in the system database and flagged for review. The confidence rating for the affected area is updated once the report is verified. Initiated by: Utility Technician Triggering Event: Utility Technician discovers infrastructure in the field that is not reflected on the current SubterraView map. Additional Actors: City Infrastructure Database, Utility Database, Main Database
Sequence of Events: 1. The Utility Technician navigates to the report submission feature within the SubterraView application. a. The system displays the crowdsourced report form. 2. The Utility Technician enters the infrastructure location, type, description, and any supporting images and submits the completed report. a. The system stores the submission, flags it as pending review, and compares it against existing infrastructure records from the City Infrastructure Database. 3. The AI validation engine reviews the submission against existing infrastructure records.

a. The system updates the confidence rating for the affected area based on the submitted report. 4. The system confirms to the Utility Technician that the report has been received and is under review. a. The system will ensure the technician's report has been sent.
Alternatives: If connectivity is limited, the system will store the report locally and submit it automatically once the connection is restored. Exceptions: If the report submission fails due to a system error, the system will notify the Utility Technician and retain the draft report for resubmission. Notes: Additional Actors are not outlined in the diagram directly but instead is referenced here and visualized via. the other use cases where the databases are the initiator!

Use case ID: UC-4	Name: Receive Hazard Alert
pre-conditions: The SubterraView application is running, and push notifications are enabled on the user's device. The Hazard Sensor Network is active and connected to the system. post-conditions: The Construction Worker and Emergency Responder receive a hazard notification with the affected location highlighted in the AR interface. The alert is logged by the system. Initiated by: Hazard Sensor Network Triggering Event: A connected underground sensor detects abnormal environmental conditions such as elevated methane levels or structural instability. Additional Actors: Construction Worker, Emergency Responder	
Sequence of Events: 1. A sensor in the Hazard Sensor Network detects an abnormal environmental reading at the job site and transmits it to SubterraView. a. The system analyzes the incoming data, determines the hazard type, severity level, and affected area, and generates a hazard alert and transmits the reading to the SubterraView Hazard Monitoring System.	

2. The Construction Worker receives a push notification and device vibration alert with the hazard type and location.
 - a. The system highlights the affected underground area in the AR interface to indicate the source and location of the hazard.
3. The Emergency Responder receives a push notification with the hazard location and type.
 - a. The system logs the alert as sent and records the timestamp and hazard details for both the Construction Worker and Emergency Responder.
4. The Construction Worker and Emergency Responder acknowledge the alert within the app.
 - a. The system records the acknowledgment and marks the alert as received.

Alternatives: If push notifications are disabled on the user's device, the system will display the alert as a prominent in-app notification the next time the user opens SubterraView.

Exceptions: If the Hazard Sensor Network loses connection, the system will notify users that hazard monitoring is temporarily unavailable. If the alert system goes offline, the system will attempt to restore the connection and notify the System Administrator of the outage.

Notes: N/A

Use case ID: UC-5

Name: Send Alerts

pre-conditions: The SubterraView application is running. Push notifications are enabled on the user's device. The hazard monitoring system is active and connected.

post-conditions: Relevant users are notified of the hazard. The alert is logged with a timestamp and hazard details.

Initiated by: Alert System

Triggering Event: Any type of Hazard Detection

Additional Actors: N/A

Sequence of Events:

1. The alert system detects a hazard or incoming update from a user or internal source. a. The system analyzes the detection and determines the hazard type, severity, and affected area. 2. The alert system broadcasts the alert to all relevant users in the nearby area. a. The system delivers push notifications and vibration alerts to all affected users' devices. 3. The AR interface highlights the affected underground area to indicate the source of the hazard. a. The system logs the alert with a timestamp and hazard details.
Alternatives: If a user submitted the detection, it will appear as a crowdsourced alert visible to other nearby users. Exceptions: If the alert system goes offline, crowdsourcing will serve as the primary way for users to share and receive alerts. If the hazard sensor network loses connection, the system will notify users that hazard monitoring is temporarily unavailable. Notes: N/A

Use case ID: UC- 6	Name: Contact 811 Service
pre-conditions: The SubterraView application is running. The Construction Worker is preparing to begin excavation at a new project site. post-conditions: The Construction Worker has been directed to contact 811 and has either completed the call or accessed the 811 websites through the app. Initiated by: Construction Worker Triggering Event: The Construction Worker selects a new project site and prepares to begin digging planning. Additional Actors: N/A	
Sequence of Events: 1. The Construction Worker selects a new project site and begins excavation planning in SubterraView.	

- a. The system displays a mandatory prompt reminding the Construction Worker to contact 811 before any excavation begins.
2. The Construction Worker selects the option to contact 811 directly through the app.
 - a. The system provides the option to place a call to 811 or access the official 811 website.
3. The Construction Worker selects their preferred contact method.
 - a. The system redirects the Construction Worker to the phone's call function or the 811 websites via the device browser.
4. The Construction Worker completes the 811-service process.
 - a. The system returns the Construction Worker to SubterraView once the call or browser session is complete.

Alternatives: If no records fall below the confidence threshold during the review, the system will notify the System Administrator that all records meet the current accuracy standards, and no follow up actions are required.

Exceptions: If the system cannot access the City Infrastructure Database or Utility Database during the review, it will complete the review using available data and flag the affected areas as requiring verification once the connection is restored.

Notes: The triggering actor is listed as Time because this use case is initiated by a regularly occurring scheduled interval rather than a direct user action

Use case ID: UC-7

Name: Validate Utility Records

pre-conditions: The AI validation engine is online. New or updated infrastructure records have been received from the Utility Database or City Infrastructure Database.

post-conditions: The incoming records are validated, assigned a confidence rating, and either integrated into the infrastructure map or flagged for manual review.

Initiated by: Databases

Triggering Event: Some users submit updated infrastructure records or new records are received from the City Infrastructure Database.

Additional Actors: System Administrator

Sequence of Events:

1. The system detects incoming infrastructure records from the Utility Database or City Infrastructure Database.
 - a. The system queues the incoming records for processing.
2. The AI validation engine retrieves the incoming records and cross references them against existing data.
 - a. The system identifies any conflicts or discrepancies between the incoming records and existing data.
3. The AI validation engine assigns a confidence rating to each incoming record.
 - a. Records that meet the confidence threshold are automatically integrated into the infrastructure map.
4. Records that fall below the confidence threshold are flagged and sent to the System Administrator for manual review.
 - a. Manual review to approve or reject records.

Alternatives: If all incoming records pass validation without conflicts, the system will complete the update cycle automatically without requiring System Administrator input.

Exceptions: If the database goes offline during an update cycle, the system will hold all pending records in a queue and resume the update process once the connection is restored. The System Administrator will be notified of any interruption.

Notes: N/A

Use case ID: UC-8

Name: Access Emergency Mode

pre-conditions: The SubterraView application is running on the Emergency Responder's device. The system has infrastructure data available for the requested location.

post-conditions: The Emergency Responder has access to a prioritized underground infrastructure map showing utility lines, confined spaces, and identified hazard zones for the incident location.

Initiated by: Emergency Responder

Triggering Event: An underground incident occurs and the Emergency Responder arrives at the scene requiring immediate infrastructure information.

Additional Actors: City Infrastructure Database, Hazard Sensor Network

Sequence of Events:

1. The Emergency Responder opens SubterraView and activates emergency mode.
 - a. The system prompts the Emergency Responder to enter or confirm the incident location.
2. The system retrieves all available infrastructure data for the incident area from the City Infrastructure Database and cross references with any active hazard alerts from the Hazard Sensor Network.
3. The system generates a prioritized underground infrastructure map highlighting utility lines, confined spaces, and active hazard zones.
 - a. The system displays the emergency map on the Emergency Responder's device with the highest risk areas clearly marked.
4. The Emergency Responder uses the map to determine safe entry points and identify potential danger zones.

Alternatives: If the incident location cannot be confirmed via GPS, the Emergency Responder may manually enter the address or coordinates.

Exceptions: If infrastructure data is unavailable or incomplete for the incident location, the system will notify the Emergency Responder and display whatever data is available with a low confidence indicator. If the system loses network connectivity during an active emergency session, it will retain the last loaded map on the device until the connection is restored.

Notes: N/A

Use case ID: UC- 9

Name: Review Data Accuracy Report

pre-conditions: The SubterraView system is online. The System Administrator is logged into the administrative interface.

post-conditions: The System Administrator receives a data accuracy report identifying infrastructure records with low confidence ratings or outdated verification status.

Initiated by: System Administrator

Triggering Event: Manual review or any time of review

Additional Actors: Databases

Sequence of Events:

5. The System Administrator initiates a data accuracy review.
 - a. The system scans all stored infrastructure records and identifies records.
6. The system identifies records that have not been verified or updated.
 - a. The system gets the flagged records into a report.
7. The system delivers the report to the System Administrator with a summary of affected areas and recommended actions.
 - a. The administrator carries out the best actions based on the report.

Alternatives: If no records fall below the confidence threshold, the system will notify the System Administrator that all records meet current accuracy standards.

Exceptions: If the system cannot access the databases, it will halt until proper connection is established.

Notes: N/A

Use case ID: UC-10

Name: Update Internal Records

pre-conditions: The AI validation engine and system databases are online. New batch of data must be needed to upload or update.

post-conditions: Records Updated.

Initiated by: System Administrator

Triggering Event: Databases checked for an update

Additional Actors: City Infrastructure Database, Hazard Sensor Network

Sequence of Events:

1. The system detects new or updated records during an internal database check.
 - a. The system queues the records for review and checks them against existing database entries for duplicates or conflicts.
2. Records with identified conflicts are flagged and sent to the System Administrator for manual review.
 - a. The system identifies new entries and flags any conflicts for manual review.
3. Manual review or the system writes approved update entries.
 - a. The system logs any changes.

Alternatives: N/A

Exceptions: If the database goes offline during an update cycle, the system will hold all records and resume once connection is restored.

Notes: N/A

Use case ID: UC- 11

Name: Store Data (Infrastructure, User, Field)

pre-conditions: The database system and main database are online. Validated data is ready for storage.

post-conditions: All infrastructure, user, and field data is stored properly and accessible to other internal components.

Initiated by: Database

Triggering Event: Any new entry into the database from user submissions, field updates, or internal sources.

Additional Actors: N/A

Sequence of Events:

1. New data entry is detected from user submissions, field updates, or internal sources.
 - a. The system queues the incoming data for storage
2. System Admins or the AI engine verify the incoming data before storage.
 - a. The system checks for any duplicate or conflicting entries in the database.
3. The database system stores the verified data in the appropriate category.
 - a. The system confirms database is up to date

Alternatives: N/A

Exceptions:

- If the database goes offline, all incoming data will be queued until the connection is restored.
- If any problems are detected, the System Administrator will be notified for manual review.

Notes: N/A

Use case ID: UC- 12	Name: Cross-Validate/ Cross-References Data
pre-conditions: The SubterraView application is running. The AI engine is online. New or updated infrastructure records have been received from a data source. post-conditions: Incoming records are validated, assigned a confidence rating, and either integrated into the infrastructure map or flagged for manual review. Initiated by: AI Engine Triggering Event: User requests site information or new records are uploaded into the system. Additional Actors: N/A	
Sequence of Events: 1. New data is uploaded to the system. a. The system processes and retrieves all relevant data for that location. 2. The AI engine cross references data from multiple sources such as public records, GIS data, and user submitted updates. a. The system analyzes the data and identifies any issues or conflicts. 3. The AI engine assigns confidence ratings to each record based on the level of agreement across data sources. a. The system integrates approved records into the infrastructure map and gets rid of any records that do not meet the confidence threshold.	
Alternatives: If a record falls below the minimum confidence threshold, it will be flagged for manual review. Exceptions: If the AI engine goes offline, the system will notify the user and stop any data processing until it is back online. Notes: N/A	

10 Functional Requirements

F-1 – User Authentication

Description: The system must present a form for users to create an account and log in by selecting their account type, which determines their access level within SubterraView.

Rationale: SubterraView serves different user groups with different needs. Allowing users to specify their account type ensures the proper features as well as organization.

Fit Criterion: 100% of accounts must have a defined account type and access level upon creation before getting access.

Acceptance Tests: 10a1: User Authentication

F-2 – AR Visualization Display

Description: The system must provide a way to display a visual overlay of underground infrastructure for a given site, allowing users to see what is beneath the surface through their device screen.

Rationale: If there is no means to view this data, then there is no way a user will be able to visualize what they want which can impact their original plans.

Fit Criterion: The system shall display a complete underground infrastructure overlay for any confirmed site location. The visualization must be accessible to all user types within 10 seconds of request.

Acceptance Tests: 10a2: AR Display

F-3 – Location Detection

Description: The system shall provide a way to access user location either through GPS detection or through a user-friendly form.

Rationale: Most features within SubterraView are location dependent. Without an accurate site location, the system cannot retrieve or display the correct underground data for the user.

Fit Criterion: Location detection shall take no more than 5 seconds to find through GPS. If manual form is filled out, the system shall process that within 10 seconds.

Acceptance Tests: 10a3: Location Detection

F-4 – Hazard Alerts

Description: The system shall notify relevant users of any hazards detected through sensors at or near their site location in real time.

Rationale: Hazard information is critical for workers and emergency responders operating near underground infrastructure. Quick alerts allow users to respond appropriately and avoid potential harm.

Fit Criterion: When new detection occurs, alerts must be sent out within 5 seconds.

Acceptance Tests: 10a4: Hazard Alerts

F-5 – Field Report Submission

Description: The system shall provide a way to submit field reports containing new infrastructure findings, corrections, or warnings discovered on site.

Rationale: During a project, users may observe infrastructure that is not yet documented. Providing a way to submit reports keeps SubterraView's data up to date.

Fit Criterion: 100% of field reports shall be stored and flagged for review. Users shall receive confirmation once their report has been successfully submitted.

Acceptance Tests: 10a5: Field Report

F-6 – 811 Service

Description: The system must provide a means to get in touch with 811. This can be a direct call to 811 or just a way to get to their website.

Rationale: As stated previously, 811 is a federal mandate and is required before any excavation at a site. This ensures that the users have a way to get any help or information regarding the 811 service.

Fit Criterion: Calling 811 services should automatically ring the correct 811 number for 100% of users who choose to call. The website option should automatically be pulled up but will also depend on the user's network strength.

Acceptance Tests: 10a6: 811 Test

F-7 – Emergency Mode

Description: The system must provide emergency responders with a dedicated emergency mode that displays a prioritized underground infrastructure map highlighting relative information such as utility lines.

Rationale: Emergency responders might not need the same overview as a regular individual or construction worker, so having a more focused output will allow for easier use for responders.

Fit Criterion: Emergency mode shall activate within 2 seconds of being launched and must display an infrastructure map with any hazard zones clearly marked.

Acceptance Tests: 10a7: Emergency Mode Test

F-8 – AI Data Validation and Records Management

Description: The system shall provide an AI engine that cross-references infrastructure records from multiple data sources, assigns confidence ratings to each record, and stores approved records in the database. The system shall also generate reports for the System Administrator.

Rationale: The integrity of SubterraView's infrastructure data depends on continuous validation and accurate storage. Without an automated engine to cross-reference sources and flag inconsistencies, the AR visualizations and hazard alert the system produces cannot be trusted.

Fit Criterion: 100% of incoming records must be assigned a confidence rating before integration. Records falling below the minimum confidence threshold must be flagged and delivered to the System Administrator.

Acceptance Tests: 10a8: Validation Test

11 Data Requirements

D-1 - Infrastructure Records

Description: The system shall store infrastructure records for each site, including utility type, location coordinates, depth, and confidence rating.

Rationale: Infrastructure records are the core details SubterraView relies on to generate accurate visualizations. Without complete and structured records, the AR system cannot produce reliable output.

Fit Criterion: 100% of infrastructure records that are stored in the database shall include utility type, location coordinates, depth estimate, and a confidence rating.

Acceptance Tests: 11a1: Infrastructure Records

D-2 – User Account

Description: The system shall store user account data including name, role, and access level for each user.

Rationale: User accounts are needed to manage access control and ensure each user group has the proper level of access within SubterraView.

Fit Criterion: 100% of registered user accounts must have a defined role and access level when creating an account.

Acceptance Tests: 11a2: User Account Details

D-3 - Field Update Report

Description: The system shall store all crowdsourced field reports submitted by users, including infrastructure location, type, description, and the time of submission.

Rationale: Field reports provide real time details that keep SubterraView's infrastructure data current and accurate.

Fit Criterion: 100% of submitted field reports shall be stored and eventually processed by AI Engine or a System Administrator.

Acceptance Tests: 11a3: Field Update Report Processed

D-4 - Hazard Alert Record

Description: The system shall store all hazard alerts generated for a given site, including hazard type, risk level, location, and time.

Rationale: Tracking hazard alerts per location allows SubterraView to maintain a history of incidents and issues for any given location. This could help users and System Administrators identify possible problems and make better decisions on site.

Fit Criterion: 100% of generated hazard alerts shall be logged with all required fields and remain accessible for authorized users of the affected site.

Acceptance Tests: 11a4: Hazard Alert

D-5 - Data Format Standards

Description: The system shall require all incoming data to follow proper formats. GIS and infrastructure data will follow standard compatible formats. GPS/location data will use latitude and longitude. Images attached to field reports can be accepted in JPEG or PNG format. Documents and reports shall be in PDF format.

Rationale: Having consistent data formatting ensures the AI engine can properly process and cross reference data from multiple sources without errors or conflicts.

Fit Criterion: Any data entry that does not match the defined format will be rejected by the system. Only properly formatted data will be accepted and stored in the database.

Acceptance Tests: 11a5: Standard Data Format

12 Performance Requirements

12a Speed and Latency Requirements

P-1 - AR Visualization Load Time

Description: The system should generate and display the AR underground infrastructure elements within a reasonable time after the user confirms their site location.

Rationale: Field workers and emergency responders need quick access to underground infrastructure data. Slow load times can cause delays that affect safety and productivity on site.

Fit Criterion: The AR visualization must load and display within 5 seconds for 90% of requests. There should not be any visualizations that take > 15 sec.

Acceptance Tests: 12a1: AR Visualization Load Time

P-2 - Hazard Alert Time

Description: The system should deliver hazard alerts to all relevant users within a reasonable time after a hazard is detected or reported.

Rationale: In an emergency, delayed alerts can put workers and responders at risk. Real time alert delivery is critical to user safety.

Fit Criterion: Hazard alerts should notify all relevant users within 5 seconds of detection/report for 90% of cases. No alert delivery shall take longer than 10 seconds.

Acceptance Tests: 12a2: Hazard Alert Delivery Time

P-3 - Data Processing Time

Description: The system should complete the AI engine data cross referencing/validation process within a reasonable time after a site location is requested.

Rationale: Users expect a responsive experience when pulling up site data. Long processing times can cause user uncertainty in the app.

Fit Criterion: The AI engine shall complete data cross referencing and return validated results within 10 seconds for 90% of requests. New locations with older data or weak data may take no longer than 25 seconds.

Acceptance Tests: 12a3: Data Processing Time

12b Precision or Accuracy Requirements

P-4 – AI Engine Accuracy

Description: The AI engine shall maintain a minimum confidence rating of 97% across all data validation, cross referencing, and infrastructure record processing tasks.

Rationale: SubterraView heavily relies on the model being there to clean/process data as well as be involved in many internal processes. The stronger, the more confident the output of the whole app will be.

Fit Criterion: The model must have a confidence measure of $\geq 97\%$. Any updates to the engine such as new deployments or fine tuning must still have a $\geq 97\%$ after updates.

Acceptance Tests: 12b1: AI Engine Accuracy

12c Capacity Requirements

P-5 - Simultaneous Users

Description: The system shall support multiple users requesting, visualizing, and submitting infrastructure data at the same time without any issues in performance.

Rationale: Many projects run simultaneously across the country. Allowing many users at different parts of the country is important to the app's success. Usage rates will vary based on season and other external factors.

Fit Criterion: The system shall support a minimum of 10,000 users across features like: requesting, visualizing, and updating without any issues.

Acceptance Tests: 12c1: Number of Concurrent Users

P-6 - Data Storage Capacity

Description: The system shall be able to store large amounts of data including infrastructure records, field reports, hazard alert logs, and any associated media such as images and documents.

Rationale: SubterraView will have more and more items in the database as time goes on from records, requests, and even hazards. The system must be able to handle this large amount of data since this is all relevant information that must be stored.

Fit Criterion: The system shall support over 1 million entries and over 4 TB of media in storage that accounts for scaling without requiring manual intervention or causing performance issues.

Acceptance Tests: 12c2: Capacity Test

13 Dependability Requirements

13a Reliability Requirements

Dep-1 - AI Engine Reliability

Description: The system shall have a reliable AI engine that remains online and functional during active use of SubterraView.

Rationale: The overall internal system of records, validation, referencing all falls on the AI engine. This is crucial and must always remain operational.

Fit Criterion: The AI Engine shall not fail more than once per month during active use. If there is a failure, the engine should take no longer than 10 min to boot back up.

Acceptance Tests: 13a1: AI Engine Test

Dep-2 - Data Reliability

Description: The system shall ensure that all stored data including infrastructure records, field reports, and all other data remains intact even in the event of a failure, downtime, or system update.

Rationale: Data is one of the most important driving factors for SubterraView and needs to be kept safe.

Fit Criterion: No data must be lost in the event of a fail, downtime, or new update/fix.

Acceptance Tests: 13a2: Data Reliability

13b Availability Requirements

Dep-3 - App Availability

Description: The system shall remain available for use with minimal downtime to ensure users can use it anytime.

Rationale: Field workers and emergency responders depend on SubterraView being accessible at any time.

Fit Criterion: The system shall maintain a minimum uptime of 99% every month. Any planned maintenance downtime shall be notified to users in advance. The maintenance downtime shall be resolved in a timely manner.

Acceptance Tests: 13b1: App Availability

13c Robustness or Fault-Tolerance Requirements

Dep-4 - Network Robustness

Description: The system shall continue to provide partial functionality through the app and sensors if there is weak network connectivity.

Rationale: Field workers and emergency responders operate in outdoor environments where network connectivity can be unreliable. The app must be able to handle connection losses without completely failing the user.

Fit Criterion: In low connection, the system shall only show the last cached infrastructure data for the active site. Users will be notified that they are currently in offline mode and suggested to connect to a stronger network.

Acceptance Tests: 13c1: Network Robustness.

13d Safety-Critical Requirements

Dep-5 - Construction/Utility/Public Worker Safety

Description: The system shall comply with OSHA safety standards and ensure that any workers are properly warned.

Rationale: These workers operate in high-risk environments where underground hazards such as gas leaks can cause serious injury or death.

Fit Criterion: The system shall show hazard alerts through sensor detection to any type of workers. The system shall also prompt all construction users to contact 811 before any excavation takes place to comply with 811 mandates.

Acceptance Tests: 13d: Safety Test

Dep-6 Regular Individual Safety

Description: The system shall show safety disclaimers and guidelines to casual users informing them that SubterraView is not an officially recognized service and should refer to 811 services before digging.

Rationale: Regular individuals may not be familiar with safety requirements surrounding excavation on personal projects. They must be properly informed to prevent any harm.

Fit Criterion: All individual users shall be shown a safety disclaimer upon opening the app. The system will still provide quick access to 811 for digging related inquiries.

Acceptance Tests: 13d: Safety Test

14 Maintainability and Supportability Requirements

14a Maintenance Requirements

M-1 - Maintenance

Description: The system shall be maintained by the Engineering team where they will fix bugs, resolve issues, deploy updates, and add new functionality with minimal downtime.

Rationale: As time progresses, SubterraView will have ongoing maintenance to make the app better, more efficient, reliable, and secure. This maintenance will again be done in a timely manner.

Fit Criterion: Any maintenance that is possibly time consuming such as new update, model tuning, and broken tests will be notified to users and completed within one week. Other minor maintenance should take no longer than 24-48 hours.

Acceptance Tests: 14a1: Maintenance Test

14b Supportability Requirements

M-2 - Support Team

Description: The system shall be supported by a dedicated support team within the engineering team. They will be at hand to solve real time issues with users and move any issues along that need a higher level of attention.

Rationale: Users can have issues within the app and would need help so a support team is something that they can reach out to for example to move their access level from individual to company.

Fit Criterion: All reported requests for support must be handled in 24 hours or must notify users an issue can take more time depending on request type. The more important a certain request the more priority it has.

Acceptance Tests: 14b1: Support Test

M-3 - In-App Support

Description: Any support that can be done within the app will be handled in the app. Before a user submits a request for support, users may find the help they need directly within the app.

Rationale: Minor issues and common questions should not require direct support team involvement. Providing in-app resources such as FAQs, safety guidelines, and 811 access saves time for both users and the support team.

Fit Criterion: The app shall include a dedicated support section containing FAQs, safety guidelines, and direct access to 811 services. Users will be pointed to these resources before submitting a proper support request.

Acceptance Tests: 14b2: In-App Support Test

14c Adaptability Requirements

M-4 - Software

Description: The system shall support various hand-held devices like iOS and Android. From phones to tablets since many construction companies may vary on what devices they give their workers.

Rationale: Ensuring SubterraView is available across multiple platforms removes any hardware restrictions for users, making the app more accessible.

Fit Criterion: The system shall run on iOS and Android devices from smartphones to tablets with no issues in performance.

Acceptance Tests: 14c1: Software Test

M-5 – Future Environments

Description: The system may expand beyond the United States and may be deployed in other countries and regions.

Rationale: Expanding SubterraView to other locations increases its impact, allowing more users to benefit.

Fit Criterion: Before deploying in a new location, the system must comply with all local safety standards and underground infrastructure regulations for that region.

Acceptance Tests: 14c2: Environment Test

14d Scalability or Extensibility Requirements

M-6 - Scalability

Description: The system shall be designed to scale as SubterraView grows in users, data, and supported regions over time.

Rationale: SubterraView is expected to grow from an initial user base to a much larger platform. The system must be able to handle this growth without requiring major changes to the core infrastructure.

Fit Criterion: The system shall be able to scale from an initial user base to a minimum of 1 million users and expand its database capacity without any issues in performance.

Acceptance Tests: 14d1: Scalability Test

14e Longevity Requirements

M-7 - Product Longevity

Description: SubterraView shall be designed and built to operate and be maintainable for a minimum of five years following its initial release.

Rationale: Ensuring SubterraView has a long lifespan maximizes the worth it brings for clients and customers.

Fit Criterion: The system shall remain fully operational and maintainable for a minimum of five years while maintaining the same core functionality and architecture.

Acceptance Tests: 14e1: Longevity Test

15 Security Requirements

15a Access Requirements

S-1 - Role-Based Access Control

Description: The product shall have a role-based access control system, where different user roles have access only to the data they are permitted to access.

Rationale: Sensitive infrastructure and hazard data must be protected, and users should only access information necessary for their responsibilities to prevent misuse.

Fit Criterion: 100% of the system functions and data shall be accessible only to authorized roles as defined in the Stakeholders section.

Acceptance Tests: 15a1: Role-Based Access Control

15b Integrity Requirements

S-2 - Infrastructure Data Integrity Protection

Description: The product shall protect infrastructure and hazard data from unauthorized modification.

Rationale: SubterraView relies on accurate infrastructure and hazard data to ensure user safety. If incorrect or corrupted data was used, it can increase the risks.

Fit Criterion: 100% of infrastructure and hazard data shall only be modified by authorized users.

Acceptance Tests: 15b1: Data Integrity Protection

15c Privacy Requirements

S-3 - Protection of User and Location Data

Description: The product shall protect any personal or sensitive user data from unauthorized access.

Rationale: SubterraView may collect user-related data such as location and submitted reports. Protecting this information is important to maintain user trust.

Fit Criterion: 100% of sensitive user data shall only be accessible to authorized users.

Acceptance Tests: 15c1: Data Privacy Protection

15d Audit Requirements

S-4 - System Activity Audit Logging

Description: The product shall maintain a log of user actions, including data updates, report submissions, and access to sensitive information.

Rationale: Maintaining an audit trail ensures accountability and allows tracking of who performed specific actions within the system.

Fit Criterion: 100% of critical system actions shall be recorded with user ID, timestamp, and action details.

Acceptance Tests: 15d1: Activity Audit Log

15e Immunity Requirements

S-5 - Protection Against Malicious Software

Description: The product shall be protected against malicious software such as viruses, malware, and unauthorized programs that may affect system functionality or data.

Rationale: SubterraView processes critical infrastructure and hazard data.

Protection against malicious software is necessary to ensure system reliability.

Fit Criterion: The system shall prevent malicious activities against it and pass standard security scans with no critical vulnerabilities detected.

Acceptance Tests: 15e1: Malware Protection

16 Usability and Humanity Requirements

16a Ease of Use Requirements

UH-1 - Rapid AR Infrastructure Interpretation

Description: If a user chooses to use the AR feature, the product should ensure users quickly understand underground infrastructure through the AR interface using clear visual overlays, color coding, and labels.

Rationale: Construction workers and emergency responders must interpret underground data in real time. Any type of delay or confusion can lead to safety risks or operational inefficiencies.

Fit Criterion: At least 90% of construction workers and emergency responders shall correctly interpret infrastructure types and locations within 5 seconds of opening the AR view.

Acceptance Tests: 16a1: AR Interpretation Speed Test

UH-2 - Minimal Interaction for Core Tasks

Description: For any user, the product should allow users to access core features (AR view, hazard alerts, report submissions) with minimal interaction steps.

Rationale: Users of SubterraView, particularly field workers, often operate in physically demanding and time-sensitive environments and cannot navigate complex interfaces while working.

Fit Criterion: At least 85% of users across all user groups shall complete core tasks within 4 interactions without assistance.

Acceptance Tests: 16a2: Minimal Interaction For Task Completion

UH-3 - Low Error Rate in Hazard Response

Description: The product must minimize user errors when responding to hazard alerts and interpreting hazard zones.

Rationale: Incorrect interpretation of hazards (e.g., gas leaks, unstable ground,) can lead to severe safety incidents.

Fit Criterion: User error rate in hazard identification and response shall be less than 2%.

Acceptance Tests: 16a3: Hazard Response

UH-4 - Immediate and Clear System Feedback

Description: For user actions and system alerts, the product shall provide immediate visual, auditory, or haptic feedback.

Rationale: Construction sites can be extremely noisy and chaotic environments. Users must receive clear confirmation that actions (e.g., report submission, hazard detection) have been successfully processed.

Fit Criterion: 100% of user actions shall generate feedback within 2 seconds, and hazard alerts shall include at least two feedback methods (e.g., visual + vibration).

Acceptance Tests: 16a4: Feedback Response Time.

UH-5 - Ease of Use for Low-Experience Users

Description: The product shall be usable by individuals with minimal technical experience, including casual users performing personal projects.

Rationale: SubterraView includes users with varying levels of technological expertise, from trained professionals to general individuals.

Fit Criterion: At least 80% of inexperienced users shall successfully complete basic tasks (viewing, infrastructure, identifying hazards) within 2 minutes without prior training.

Acceptance Tests: 16a5: Inexperienced User Task Completion

16b Personalization and Internationalization Requirements

UH-6 - Measurement Unit Configuration

Description: The product shall allow users to configure measurement units for distance and depth (e.g., feet or meters) within the application settings.

Rationale: Users may operate in environments that use different measurement systems. Allowing unit customization ensures that users can understand infrastructure data accurately without conversion errors.

Fit Criterion: 100% of users shall be able to select and apply their preferred measurement unit and all displayed values shall update immediately after selection.

Acceptance Tests: 16b1: Unit Configuration Functionality

UH-7 - Design Preference Configuration

Description: The product shall allow users to configure personal preferences such as display settings (e.g., colors) and notification preferences, and retain these settings across sessions.

Rationale: Allowing users to personalize the system improves usability and ensures that the application aligns with individual working conditions and preferences.

Fit Criterion: All configured user preferences shall persist across sessions with no loss of data.

Acceptance Tests: 16b2: Design Configuration

UH-8 - Notification Preference Configuration

Description: The product shall allow users to configure how they receives alerts, including visual, auditory, and haptic notification options.

Rationale: Different users operate in different environments (e.g., noisy construction sites), requiring flexible notification methods to ensure alerts are noticed.

Fit Criterion: Users shall be able to enable or disable notification types, and selected configurations shall be applied to 100% of system alerts.

Acceptance Tests: 16b3: Notification Configuration

UH-9 - Language Support

Description: The product shall provide support for English as the default language and allow for future expansion to additional languages.

Rationale: Supporting multiple languages increases accessibility and ensures the system can be used by a broader range of users.

Fit Criterion: All system text shall be consistently displayed in English, and the system architecture shall support integration of additional languages without major redesign.

Acceptance Tests: 16b4: Language Support.

16c Learning Requirements

UH-10 - Immediate Basic Usability

Description: The product shall be usable by first-time users to perform basic tasks (e.g., viewing AR infrastructure and identifying hazards) without requiring prior training or reference to documentation.

Rationale: Users that use the system without formal training should be able to immediately do their tasks without much trouble.

Fit Criterion: At least 80% of first-time users shall successfully complete basic tasks within 2 minutes of initial use without external assistance.

Acceptance Tests: 16c1: Basic Usability

16d Understandability and Politeness Requirements

UH-11 - Intuitive Visual Representation of Infrastructure

Description: The product shall be clear, standardized visual elements (e.g., color coding, labels, and symbols) to represent underground infrastructure and hazards in the AR interface.

Rationale: Users must be able to immediately understand what they are viewing without needing to interpret complex or unfamiliar representations, especially in high-risk environments.

Fit Criterion: At least 90% of users shall correctly interpret the meaning of visual elements (e.g., pipe types, hazard zones) without additional explanation during usability testing.

Acceptance Tests: 16d1: Visual Interpretation

UH-12 - Familiar Language

Description: The product shall use terminology and messaging that is understandable to any user.

Rationale: Users should not be required to understand system internals unrelated to their tasks.

Fit Criterion: At least 75% of users shall report that system messages and labels are clear and easy to understand without clarification.

Acceptance Tests: 16d2: User Comprehension

16e Accessibility Requirements

UH-13 - Visual Accessibility

Description: The product shall ensure that visual elements, including AR overlays and hazard indicators, are distinguishable by users with common visual impairments.

Rationale: The system relies heavily on visual information. Users with color vision impairments or reduced eyesight must still be able to interpret infrastructure and hazard data accurately.

Fit Criterion: At least 95% of visual elements shall remain distinguishable when tested under common color blindness conditions (e.g., red-green color deficiency), and all critical information shall include both color and non-color indicators.

Acceptance Tests: 16e1: Colorblind Accessibility

UH-14 - Alert Accessibility

Description: The product shall provide alerts through multiple modalities, including visual, auditory, and haptic (vibration) feedback.

Rationale: Users may have hearing or visual impairments or may operate in environments where one type of alert is ineffective.

Fit Criterion: 100% of critical alerts shall be delivered through at least two different modalities, ensuring accessibility under varying conditions.

Acceptance Tests: 16e2: Multi-Modal Alert Test

16f User Documentation Requirements

UH-15 - User Guide and In-App Help Documentation

Description: The product shall provide a user guide and integrate in-app help system that explains how to use core features.

Rationale: Users have varying levels of technical experience, providing accessible documentation ensures that all users can understand how to operate the system effectively without requiring extensive training.

Fit Criterion: At least 90% of users shall be able to locate and use relevant help documentation to complete a task within 2 minutes of accessing the help system.

Acceptance Tests: 16f1: Help System Accessibility

16g Training Requirements

UH-16 - Minimal Training for Use

Description: The product shall require minimal training for users.

Rationale: SubterraView is designed to be intuitive and user friendly, however, structured training may be beneficial for field workers to fully utilize features such as hazard interpretation.

Fit Criterion: After completing a training session (less than 1 hour long), at least 90% of trained users shall successfully perform core operations.

Acceptance Tests: 16g1: Post-training Performance

17 Look and Feel Requirements

17a Appearance Requirements

L-1 - High-Visibility AR Overlay Design

Description: The product shall use high-contrast colors, clear labels, and consistent visual elements to display underground infrastructure and hazard zones.

Rationale: Users operate in outdoor environments with varying lighting conditions. Clear and high-visibility visuals are necessary to ensure infrastructure and hazards are distinguishable.

Fit Criterion: At least 95% of users shall be able to clearly distinguish infrastructure types and hazard zones under varying conditions.

Acceptance Tests: 17a1: Visibility of AR

L-2 - Consistent Visual Design

Description: The product shall maintain consistent use of colors, symbols, and layout across all screens.

Rationale: Consistency in appearance helps users quickly recognize system elements and reduces confusion when navigating between features.

Fit Criterion: 100% of system screens shall follow a defined visual design standard with no inconsistencies.

Acceptance Tests: 17a2: UI Consistency

17b Style Requirements

L-3 - Professional and Safety-Oriented Style

Description: The product shall present a professional, safety-focused interface that emphasizes clarity and reliability.

Rationale: SubterraView is used in high-risk situations where the users must trust the system. A professional and safety-oriented design reinforces confidence.

Fit Criterion: At least 85% of users shall report that the system appears professional and trustworthy.

Acceptance Tests: 17b1: Professionalism Perception

18 Operational and Environmental Requirements

18a Expected Physical Environment

O-1 - Outdoor Construction Site Environment

Description: The product shall be usable in outdoor environments such as construction sites, including conditions with varying weather and lighting.

Rationale: SubterraView is primarily used by construction workers and utility crews in outdoor field environments where lighting and weather conditions are not predictable.

Fit Criterion: The product shall remain fully functional in all weather conditions.

Acceptance Tests: 18a1: Outdoor Usability

O-2 - High-Noise and Active Work Environments

Description: The product shall be usable in environments with high levels of noise and physical activity.

Rationale: Construction environments are often loud and chaotic, requiring the system to function effectively.

Fit Criterion: All critical alerts shall remain perceivable in noisy environments.

Acceptance Tests: 18a2: High-Noise Usability

18b Requirements for Interfacing with Adjacent Systems

O-3 - Integration with External Data Systems

Description: The product shall interface with external systems including city infrastructure databases, utility databases, GPS services, and hazard sensor devices.

Rationale: SubterraView heavily relies on accurate and up-to-date external data sources to generate reliable infrastructure maps and hazard alerts.

Fit Criterion: The system shall successfully receive and process data from connected systems with a data update latency of less than 2 seconds.

Acceptance Tests: 18b1: External Data Systems.

18c Productization Requirements

O-4 - Mobile Application Distribution

Description: The product shall be distributed as a mobile application available on standard app distribution platforms (e.g., iOS App store and Android).

Rationale: SubterraView is intended for widespread use by construction workers, emergency responders, and general users. Easy distribution and installation ensure accessibility and adoption without requiring setup.

Fit Criterion: At least 95% of users shall successfully download, install, and launch the application within 5 minutes.

Acceptance Tests: 18c1: Application Installation

18d Release Requirements

O-5 - Scheduled Release Updates

Description: The product shall follow a scheduled release cycle for updates, including bug fixes, new features, and performance improvements.

Rationale: Regular updates ensure the system remains accurate and reliable as new data, features, and improvements are introduced.

Fit Criterion: The system shall provide at least one update per quarter, and 100% of existing core features shall remain functional after each release.

Acceptance Tests: 18d1: Post-release Validation

19 Cultural and Political Requirements

19a Cultural Requirements

C-1 - Understandable Symbols

Description: The product shall use symbols/terminology that are commonly recognized within construction, utility, and emergency response industries.

Rationale: Users from different backgrounds must be able to interpret infrastructure and hazard information without confusion.

Fit Criterion: At least 90% of users across different user groups shall correctly interpret system symbols and terminology.

Acceptance Tests: 19a1: Symbol Interpretation

C-2 - Adaptability to Regional Infrastructure Standards

Description: The product shall support adaptation to regional infrastructure conventions.

Rationale: Users from different regions will need SubterraView to be adaptable to their infrastructure conventions.

Fit Criterion: The system shall allow configuration to match regional infrastructure standards with 100% consistency.

Acceptance Tests: 19a2: Regional Configuration

19b Political Requirements

C-3 - Stakeholder Access

Description: The product shall provide full access and control to authorized stakeholders such as system administrators.

Rationale: Stakeholders such as contractors and utility organizations require appropriate control over data.

Fit Criterion: 100% of authorized stakeholder roles shall have access to their designated system functions.

Acceptance Tests: 19b1: Stakeholder Permissions

20 Legal Requirements

20a Compliance Requirements

Leg-1 - Compliance with Regulatory Requirements

Description: The product shall comply with all applicable regulatory requirements and provide appropriate access to authorized stakeholders.

Rationale: SubterraView operates within regulated infrastructure and construction environments where they must adhere to policies and stakeholder expectations.

Fit Criterion: The system shall be reviewed and approved by relevant regulatory guidelines, passing 100% of their compliance checks.

Acceptance Tests: 20a1: Regulatory Compliance

20b Standards Requirements

Leg-2 - Industry Data Standards

Description: The product shall follow established industry standards for relevant data and infrastructure mapping.

Rationale: SubterraView integrates data from multiple external sources such as city databases and utility providers. Following industry standards ensures compatibility and consistency.

Fit Criterion: The system shall successfully import and process relevant data in standard formats with 100% compatibility.

Acceptance Tests: 20b1: Data Standards

21 Requirements Acceptance Tests

21a Requirements – Test Correspondence Summary

Table	Requirements										
	Req 1	Req 2	Req 3	Req 4	Req 5	Req 6	Req 7	Req 8	Req 9	Req 10	Req 11
10a1	X										
10a2	X										
10a3	X										
10a4	X										
10a5	X										
10a6	X										
10a7	X										
10a8	X										
11a1		X									
11a2		X									
11a3		X									
11a4		X									
11a5		X									
12a1			X								
12a2			X								
12a3			X								
12b1			X								
12c1			X								
12c2			X								
13a1				X							
13a2				X							
13b1				X							
13c1				X							
13d				X							
14a1					X						
14b1					X						
14b2					X						
14c1					X						
14c2					X						
14d1					X						
14e1					X						
15a1						X					
15b1						X					
15c1						X					
15d1						X					
15e1						X					
16a1							X				
16a2							X				
16a3							X				
16a4							X				
16a5							X				
16b1							X				
16b2							X				
16b3							X				
16b4							X				
16c1							X				
16d1							X				
16d2							X				
16e1							X				
16e2							X				
16f1							X				
16g1							X				
17a1								X			
17a2								X			
17b1								X			
18a1									X		
18a2									X		
18b1									X		
18c1									X		
18d1									X		
19a1										X	
19a2										X	
19b1										X	
20a1											X
20b1											X

Table 5 - Requirements - Acceptance Tests Correspondence

21b Acceptance Test Descriptions

10a1 - User Authentication

Description: Checks if users can create an account and log in with the correct account type and access level.

10a2 - AR Display

Description: Checks if the system displays a complete underground infrastructure overlay for a confirmed site.

10a3 - Location Detection

Description: Checks if the system detects or accepts a site location via GPS or manual entry.

10a4 – Hazard Alerts

Description: Checks if hazard alerts are sent to relevant users within the time.

10a5 – Field Report

Description: Checks if submitted field reports are stored, flagged, and confirmed to the user.

10a6 – 811 Test

Description: Checks if the system connects users to 811 via call or website through the app.

10a7 - Emergency Mode Test

Description: Checks if emergency mode activates and displays a prioritized infrastructure map.

10a8 - Validation Test

Description: Checks if the AI engine and all relevant databases are correctly processing all information with proper confidence measures.

11a1 - Infrastructure Records

Description: Checks if all stored infrastructure records contain the required fields.

11a2 - User Account Details

Description: Checks if all user accounts have a defined role and access level upon creation.

11a3– Field Update Report Processed

Description: Checks if all submitted field reports are stored and processed correctly.

11a4 – Hazard Alert

Description: Checks if all hazard alerts are logged with the required fields and accessible to users.

11a5 – Standard Data Format

Description: Checks if data is proper format or type.

12a1 - AR Visualization Load Time

Description: Checks if the AR visualization loads within 5 seconds for testing requests.

12a2 - Hazard Alert Delivery Time

Description: Checks if hazard alerts are delivered to all relevant users.

12a3 - Data Processing Time

Description: Checks if the system processes and returns validated data within the required time limits.

12b1 - AI Engine Accuracy

Description: Checks if the AI engine maintains a minimum confidence rating of 97% across all tasks.

12c1 - Number of Concurrent Users

Description: Checks if the system supports the minimum simultaneous users without issues.

12c2 - Capacity Test

Description: Checks if the system supports the entries and media storage.

13a1 - AI Engine Test

Description: Checks if the AI engine stays operational and recovers if failed.

13a2 - Data Reliability

Description: Checks if no data is lost during a failure, downtime, or system update.

13b1 - App Availability

Description: Checks if the system maintains a minimum uptime of 99% monthly.

13c1 - Network Robustness

Description: Checks if the system displays cached data and notifies users when in offline mode.

13d - Safety Test

Description: Checks if users receive appropriate safety disclaimers and users are prompted to contact 811.

14a1 - Maintenance Test

Description: Checks if maintenance is completed within the required timeframes with proper user notification.

14b1 - Support Test

Description: Checks if all support requests are handled within 24 hours based on priority.

14b2 - In-App Support Test

Description: Checks if the app includes FAQs, safety guidelines, and 811 access.

14c1 - Software Test

Description: Checks if the system runs without issues on iOS and Android smartphones and tablets.

14c2 - Environment Test

Description: Checks if the system complies with local standards before deploying in a new region.

14d1 - Scalability Test

Description: Checks if the system scales to support many users without performance issues.

14e1 - Longevity Test

Description: Checks if the system remains operational and maintainable for a minimum of five years.

15a1 - Role Based Access Control

Description: Checks if users only have access to the data and features allowed for their role.

15b1 - Data Integrity Protection

Description: Checks if infrastructure and hazard data cannot be modified by unauthorized users.

15c1 - Data Privacy Protection

Description: Checks if sensitive user data is only accessible to authorized users.

15d1 - Activity Audit Log

Description: Checks if all important user actions are recorded with correct details.

15e1 - Malware Protection

Description: Checks if the system is protected against malicious software and passes security scans.

16a1 - AR Interpretation Speed Test

Description: Checks if users can quickly understand underground infrastructure using the AR view.

16a2 - Minimal Interaction For Task Completion

Description: Checks if users can complete main tasks using a small number of interactions.

16a3 - Hazard Response

Description: Checks if users correctly respond to hazard alerts without making mistakes.

16a4 - Feedback Response Time

Description: Checks if the system gives feedback quickly after user actions.

16a5 - Inexperienced User Task Completion

Description: Checks if new users can complete basic tasks without training.

16b1 - Unit Configuration Functionality

Description: Checks if users can switch between measurement units and see updated values.

16b2 - Design Configuration

Description: Checks if user preferences like display settings are saved and applied correctly.

16b3 - Notification Configuration

Description: Checks if users can change how they receive alerts.

16b4 - Language Support

Description: Checks if the system displays all text correctly in the selected language.

16c1 - Basic Usability

Description: Checks if first-time users can complete basic tasks quickly without help.

16d1 - Visual Interpretation

Description: Checks if users understand the meaning of AR visuals and symbols.

16d2 - User Comprehension

Description: Checks if users understand system messages and labels.

16e1 - Colorblind Accessibility

Description: Checks if visuals are clear for users with color vision issues.

16e2 - Multi-Modal Alert Test

Description: Checks if alerts are delivered using multiple methods (visual, sound, vibration).

16f1 - Help System Accessibility

Description: Checks if users can find and use help documentation easily.

16g1 - Post-training Performance

Description: Checks if users can perform tasks correctly after training.

17a1 - Visibility of AR

Description: Checks if AR visuals are clear in different lighting conditions.

17a2 - UI Consistency

Description: Checks if design elements stay consistent across the app.

17b1 - Professionalism Perception

Description: Checks if users feel the app looks professional and trustworthy.

18a1 - Outdoor Usability

Description: Checks if the system works properly in outdoor environments.

18a2 - High-Noise Usability

Description: Checks if users can still notice alerts in noisy environments.

18b1 - External Data Systems

Description: Checks if the system correctly receives and processes data from external sources.

18c1 - Application Installation

Description: Checks if users can download and install the app easily.

18d1 - Post-release Validation

Description: Checks if updates do not break existing features.

19a1 - Symbol Interpretation

Description: Checks if users understand symbols and terminology.

19a2 - Regional Configuration

Description: Checks if the system adapts to different regional settings.

19b1 - Stakeholder Permissions

Description: Checks if users have the correct access based on their role.

20a1 - Regulatory Compliance

Description: Checks if the system follows required laws and regulations.

20b1 - Data Standards

Description: Checks if the system follows industry data format standards.

III Design

22 Design Goals

Data Accuracy and Integrity

SubterraView operates in high-risk environments where incorrect infrastructure data can lead to worker injuries or utility strikes. The design should prioritize the

correctness and integrity of AI-validated infrastructure data above all other considerations. Where trade-offs arise between processing speed and data accuracy, the design should favor accuracy.

System Reliability and Fault Tolerance

SubterraView must remain operational in unpredictable outdoor field environments where connectivity and hardware conditions are inconsistent. The design should minimize single points of failure and support graceful degradation, such as serving cached infrastructure data during network loss, so that users always have access to some level of infrastructure information.

Real-Time Responsiveness

Field workers and emergency responders depend on SubterraView delivering infrastructure visualizations and hazard alerts quickly. The design should minimize processing and delivery latency across all core functions, particularly hazard alert delivery and AR visualization loading, to ensure the system remains useful in time-sensitive situations.

Safety Prioritization

Safety-critical features including hazard alerts, 811 prompts, and emergency mode must always be immediately accessible and treated as the highest priority events within the system. The design should ensure that no architectural decision reduces the visibility or delivery speed of safety-related outputs.

Scalability

SubterraView is expected to grow from an initial deployment to a large user base spanning multiple cities and regions. The design should allow cloud infrastructure, database capacity, and concurrent user support to scale independently without requiring fundamental changes to the core system architecture.

Security and Access Control

SubterraView handles sensitive infrastructure, hazard, and location data. The design should enforce role-based access control and data protection at every layer of the system, ensuring users can only access information and features appropriate to their assigned role.

Usability and Simplicity

SubterraView is used by field workers operating in physically demanding, noisy, and time-sensitive environments. The design should minimize the number of interactions required to complete core tasks and ensure the interface remains clear, readable, and responsive under real-world field conditions.

Maintainability and Longevity

SubterraView is expected to remain operational and supported for a minimum of five years following release. The design should favor modular, loosely coupled components that can be updated, replaced, or extended independently to reduce the cost and risk of future maintenance, bug fixes, and feature additions.

23 Current System Design

SubterraView is not intended to replace any single pre-existing system. The current underground infrastructure management process relies on a combination of the 811-notification service, fragmented city and utility databases, manual field verification, and independent hazard detection systems that operate without integration.

SubterraView is designed to supplement these existing processes rather than replace them.

24 Proposed System Design

24a Initial System Analysis and Class Identification

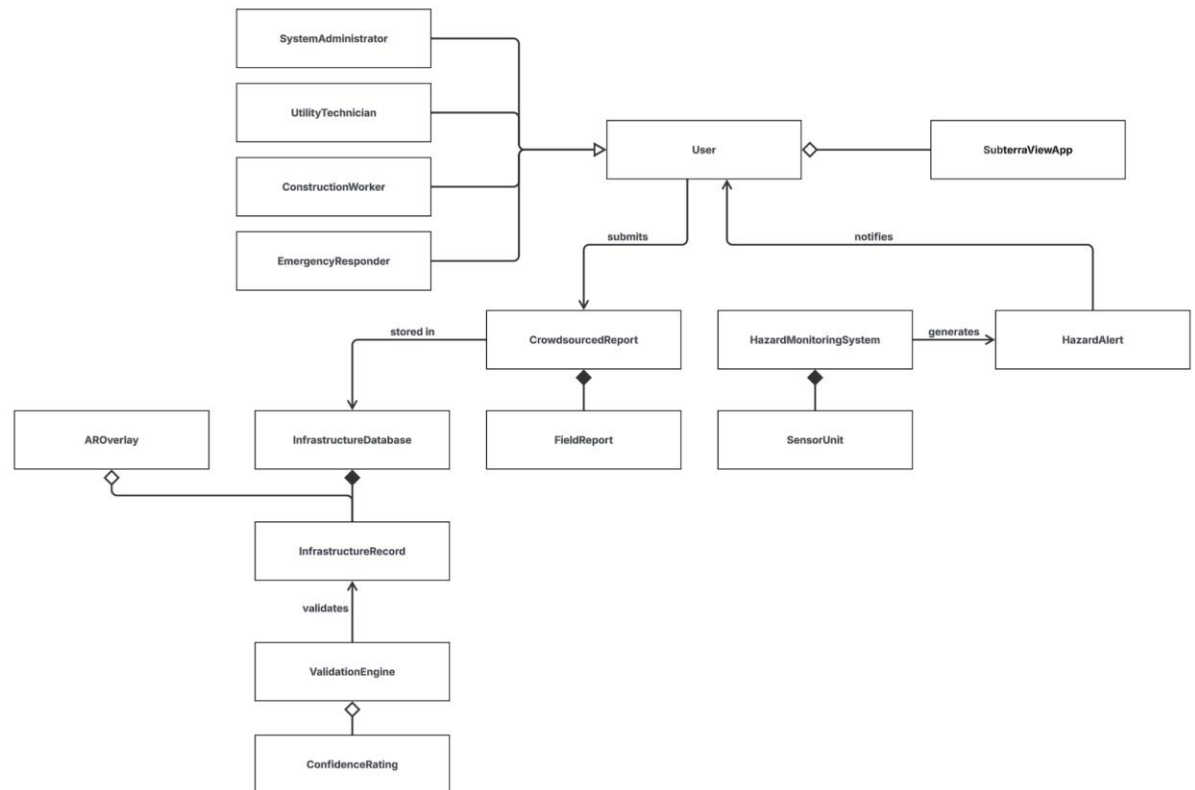


Figure 4 - Initial Class Diagram

24b Dynamic Modelling of Use-Cases

UC-1: AR Visualization

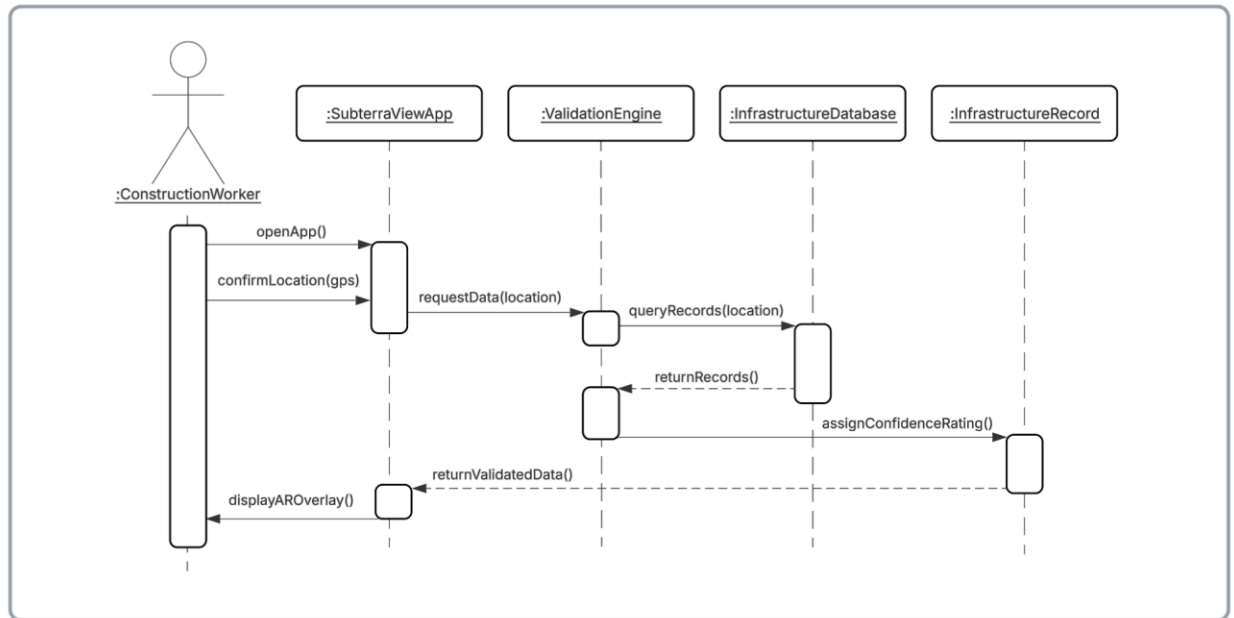


Figure 5 - Sequence Diagram UC-1

UC-3: Submit Crowdsourced Report

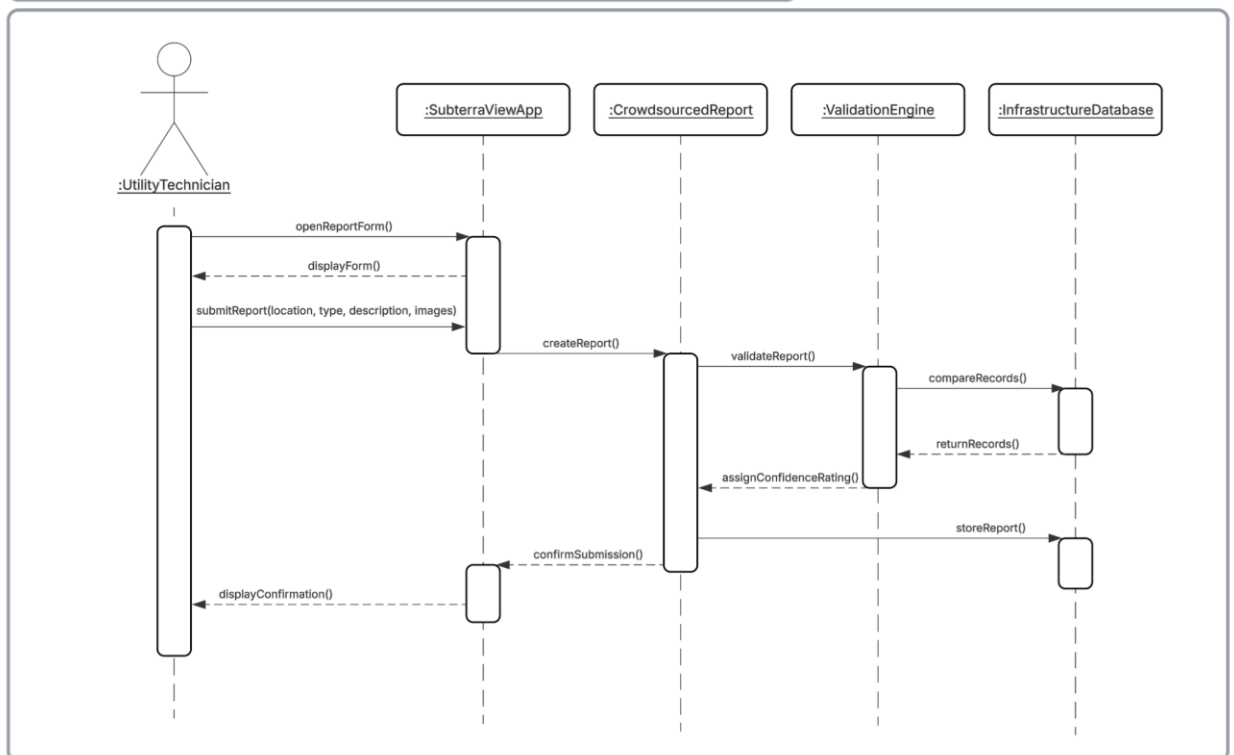


Figure 6 - Sequence Diagram UC-3

UC-4: Recieve Hazard Alert

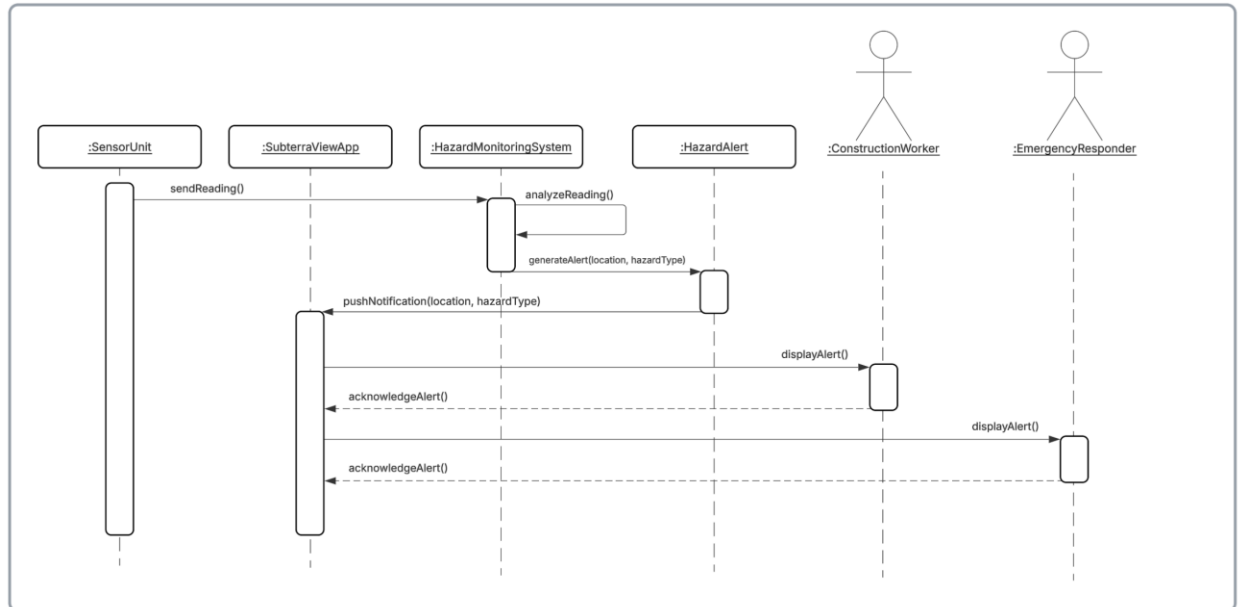


Figure 7 - Sequence Diagram UC-4

UC-7: Validation Engine

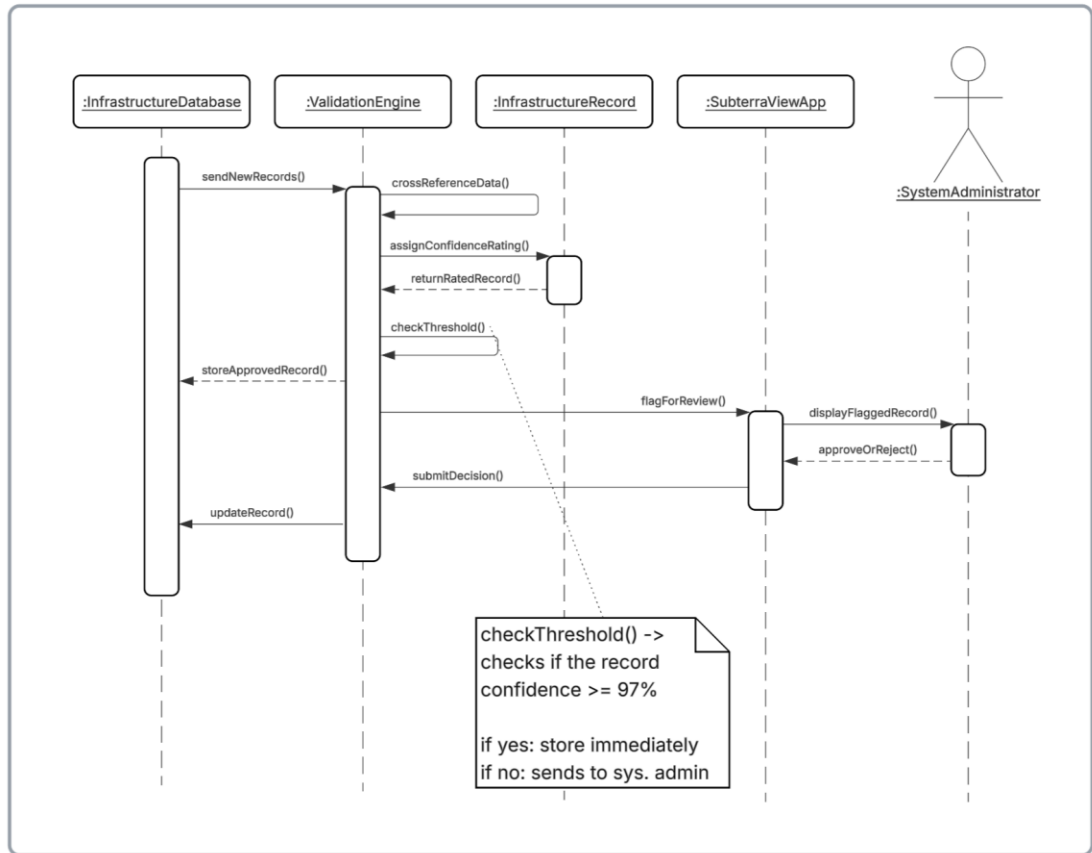


Figure 8 - Sequence Diagram UC-7

UC-8: Access Emergency Mode

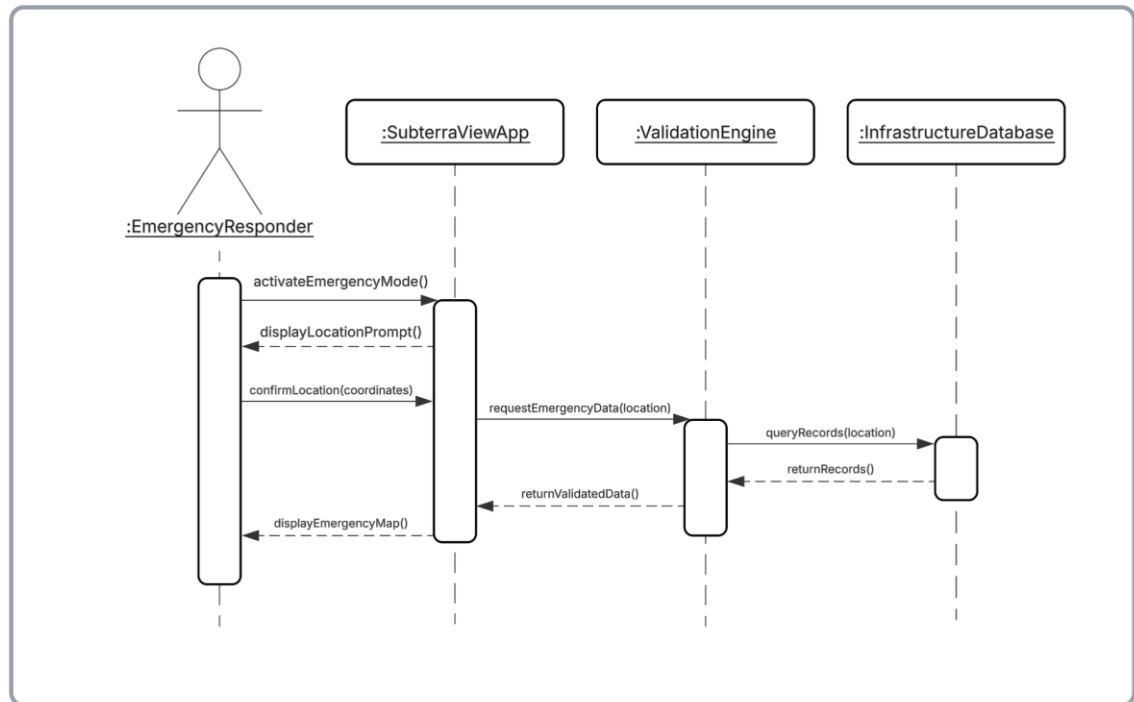


Figure 9 - Sequence Diagram UC-8

24c Proposed System Architecture

SubterraView will use a combination of Client-Server and Model-View-Controller (MVC) as its software architecture.

Client-Server is an architecture where servers provide services independently of clients, and clients may contact multiple servers. SubterraView follows this pattern in two ways. First, the mobile application running on the user's iOS or Android device acts as the client, and the cloud backend acts as the server, providing infrastructure data and validation services to any connected device. Second, the hazard sensor units deployed at job sites act as clients that continuously send environmental readings to the SubterraView server, which processes them through the HazardMonitoringSystem. This architecture was chosen because SubterraView depends on real-time communication between multiple remote devices and a central backend, which Client-Server handles naturally.

MVC separates a system into three components. The Model holds the data, the View displays it, and the Controller processes input and updates the model. In SubterraView, the Model consists of the InfrastructureDatabase and InfrastructureRecord classes which hold all infrastructure data. The View consists of the SubterraViewApp and AROverlay classes which display that data to the user. The Controller consists of the ValidationEngine and HazardMonitoringSystem which process incoming data and update the model accordingly. MVC was chosen because

it keeps the data, logic, and interface clearly separated, making the system easier to maintain and update over time.

24d Initial Subsystem Decomposition

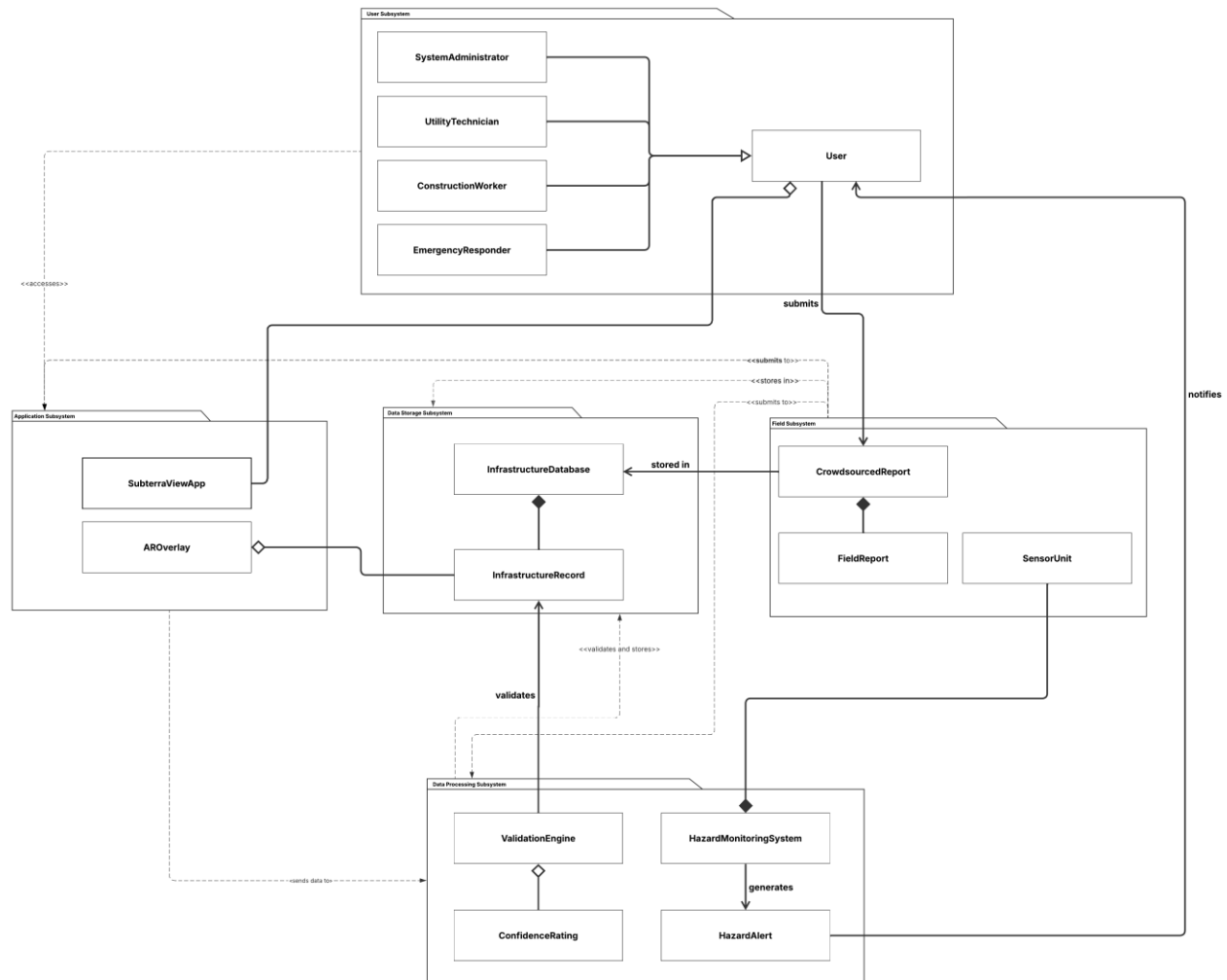


Figure 10 - Initial Subsystem Package & Class Diagram

The classes identified in Sections 24a and 24b are partitioned into five subsystems based on their shared responsibilities and relationships within the system. The subsystem decomposition diagram above illustrates how these subsystems interact with each other.

The User Subsystem represents all human actors that interact with SubterraView. It is responsible for defining the different user roles within the system and the access levels associated with each role. The four user types extend from a base User class and interact with the rest of the system through the Application Subsystem.

The Application Subsystem serves as the primary interface between the user and the rest of the system. It is responsible for displaying infrastructure visualizations, delivering hazard alerts, and handling all user input across iOS and Android

platforms. All user interactions flow through this subsystem before being passed to the appropriate backend component.

The Data Processing Subsystem is responsible for all data validation, hazard detection, and alert generation within the system. The ValidationEngine cross-references incoming infrastructure records and assigns confidence ratings, while the HazardMonitoringSystem processes sensor readings and generates hazard alerts when dangerous conditions are detected.

The Field Subsystem is responsible for collecting data from the physical environment. It handles incoming sensor readings from deployed hardware units and manages the submission and storage of crowdsourced field reports submitted by users in the field. The Data Storage Subsystem is responsible for storing and managing all validated infrastructure records within the system. It serves as the central data layer that other subsystems depend on to retrieve and persist infrastructure data.

25 Additional Design Considerations

25a Hardware / Software Mapping

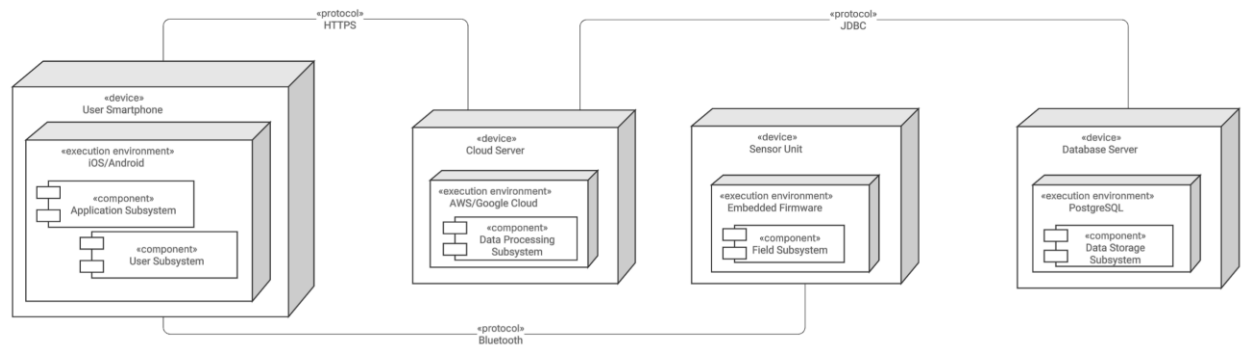


Figure 11 - Deployment Diagram

The User Smartphone runs the Application Subsystem and User Subsystem, serving as the primary interface for all user interactions. The Cloud Server hosts the Data Processing Subsystem, which handles all AI validation and hazard monitoring logic. The Sensor Unit runs the Field Subsystem, continuously collecting and transmitting environmental readings from the job site. The Database Server hosts the Data Storage Subsystem, which manages all persistent infrastructure records using PostgreSQL.

As a result of this distributed client-server mapping, two additional communication classes are required to manage data transfer between nodes. A BluetoothService class is needed to handle the real time transmission of sensor readings from the Sensor Unit to the User Smartphone over Bluetooth. A NetworkCommunicationService class is needed to manage all HTTPS communication between the User Smartphone and the Cloud Server, including sending user requests, receiving validated infrastructure data, and delivering hazard alerts back to the app. These two classes were not identified in the initial grammatical analysis but are necessary to support the distributed architecture of SubterraView.

25b Persistent Data Management

Persistent data refers to any information that must be stored when the system shuts down and restored when it starts back up again. The full data dictionary defining SubterraView's key data structures is provided in Section 7c. The most relevant items for this design phase are InfrastructureID, GeographicLocation, ConfidenceRating, and UserID, which together represent the core data that must persist across sessions.

The Data Storage Subsystem is the primary subsystem responsible for managing all persistent data in SubterraView. Within this subsystem, the InfrastructureDatabase class stores and retrieves all validated infrastructure records, user account data, and crowdsourced field reports. The InfrastructureRecord class represents each individual infrastructure element that must persist across sessions since the AR visualization depends entirely on this data being available when the app starts.

The NetworkCommunicationService class identified in Section 25a handles local caching of infrastructure data in low connectivity environments, ensuring the app can still function when the cloud server cannot be reached. On startup, the StartupManager class identified in Section 25e is responsible for restoring persistent data by loading cached records and re-establishing connections to the Data Storage Subsystem. All persistent data is stored using PostgreSQL with the PostGIS extension.

25c Access Control and Security

SubterraView handles sensitive infrastructure and location data, which requires a well-defined access control system. As defined in Section 15, the system uses role-based access control where each user role has access only to the features and data appropriate for their responsibilities. Construction Workers can view infrastructure maps and receive hazard alerts. Utility Technicians can additionally submit field reports and flag discrepancies. Emergency Responders can access emergency mode and priority infrastructure maps. System Administrators have full access to all system functions including data accuracy reviews and record management.

To implement this, an AuthenticationService class needs to be added to the design. This class is responsible for verifying user credentials at login, assigning the correct access level based on the user role, and enforcing access restrictions throughout the session. A SecurityProxy class should also be added to sit between the SubterraViewApp and the backend services, filtering requests based on the authenticated user role before they reach the Data Processing or Data Storage subsystems. This approach was identified in the lecture material as a common and effective pattern for implementing access control in distributed systems.

All data transmitted between the User Smartphone and the Cloud Server is encrypted over HTTPS as identified in the deployment diagram in Section 25a, ensuring that sensitive infrastructure and user data cannot be intercepted during transmission.

25d Global Software Control

SubterraView uses an event driven control flow as its primary method of global software control. Rather than following a fixed sequence of procedures, the system

responds to events as they occur, such as a user opening the app, a sensor detecting a hazard, or a new infrastructure record being submitted. This approach was chosen because SubterraView operates in a real time environment where hazard alerts and data updates can occur at any time and must be handled immediately regardless of what else the system is doing.

Hazard alerts are treated as the highest priority events in the system. When the HazardMonitoringSystem receives an abnormal sensor reading, it interrupts normal system operation and immediately dispatches an alert through the NetworkCommunicationService to all relevant users. All other events such as data validation requests and field report submissions are handled through a task queue managed by the Data Processing Subsystem, where tasks are processed in order of priority.

A SessionManager class needs to be added to the design to handle global control concerns such as managing active user sessions, tracking device state, and ensuring the system responds correctly when switching between online and offline modes. This class works alongside the NetworkCommunicationService to monitor connectivity and trigger the appropriate fallback behavior when the cloud server cannot be reached.

25e Boundary Conditions

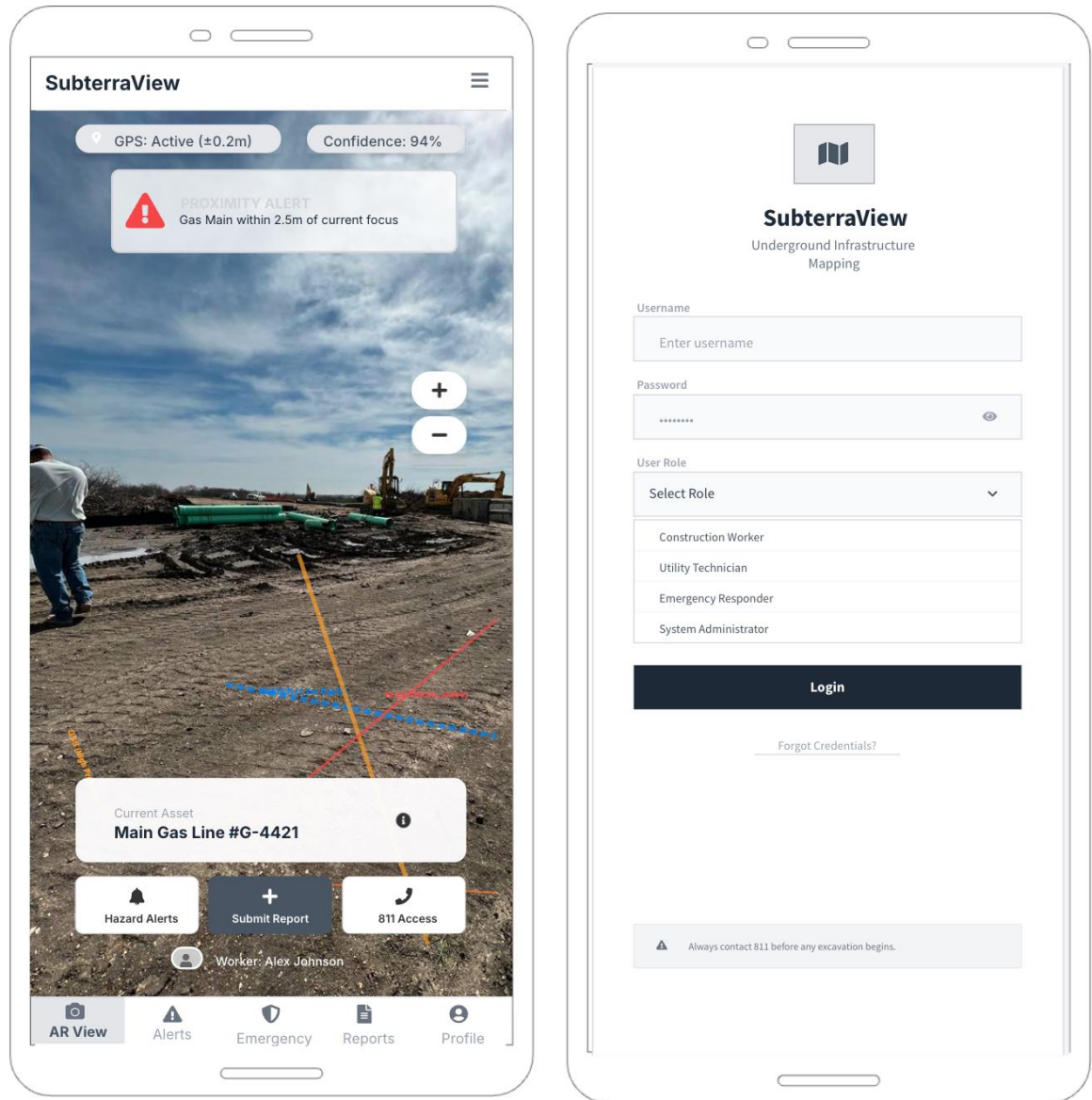
Boundary conditions refer to situations outside of normal steady state operation, including system startup, shutdown, and exception handling. These are often overlooked but are critical to ensuring SubterraView behaves reliably in real world field environments.

On startup, the system must initialize the user session, verify network connectivity, load any locally cached infrastructure data, and confirm that the Bluetooth sensor connection is active if sensors are deployed at the site. A StartupManager class needs to be added to handle this initialization sequence and ensure all required components are online before the user is allowed to interact with the system.

On normal shutdown, the system must save any unsaved user preferences, store any pending field reports locally if they have not yet been submitted, and close all active database connections cleanly. On abnormal shutdown such as an app crash or sudden loss of power, the system must ensure that no partially written records are committed to the database, protecting data integrity as required by Section 13a.

Exception handling must account for several scenarios identified in the use cases including loss of network connectivity, unavailability of the AI validation engine, failure of the Bluetooth sensor connection, and GPS unavailability. In each case the system notifies the user of the issue and falls back to the most recently cached data where possible rather than failing completely.

25f User Interface



The Login Screen provides input fields for username and password, a role selection dropdown, and a Login button. A safety disclaimer reminding users to contact 811 before any excavation begins is displayed at the bottom of the screen.

The AR View Screen serves as the primary interface for field users. It displays a live camera feed with color coded underground infrastructure overlays, a GPS status badge, a confidence rating indicator, and a proximity alert notification for nearby hazards. The bottom toolbar provides quick access to Hazard Alerts, Submit Report, and 811 Access, while a bottom navigation bar allows users to switch between the main sections of the app.

Any additional classes identified because of this interface design, such as screen navigation and form handling, are considered internal implementation details of the

SubterraViewApp class and do not require separate class definitions at this stage of the design.

25g Application of Design Patterns

The Singleton pattern is applied to the SessionManager class. Since only one active user session should exist on a device at any given time, the SessionManager is designed so that only one instance of it can be created. This prevents issues that could come up if multiple session instances were running at the same time, especially when the app is switching between online and offline modes.

The Observer pattern is applied to the HazardMonitoringSystem. When a hazard is detected, the HazardMonitoringSystem needs to notify multiple components at the same time including the SubterraViewApp, the HazardAlert class, and the NetworkCommunicationService. Using the Observer pattern allows these components to register with the HazardMonitoringSystem and get notified automatically when a hazard event occurs, without the HazardMonitoringSystem needing to know the details of each component it is notifying. This supports the real time hazard alert delivery requirements.

The Proxy pattern is applied through the SecurityProxy class identified in Section 25c. The SecurityProxy sits between the SubterraViewApp and the backend services and filters all incoming requests based on the authenticated user role before they reach the Data Processing or Data Storage subsystems. This ensures that users can only access the features and data that their role permits, directly supporting the access control requirements.

26 Final System Design

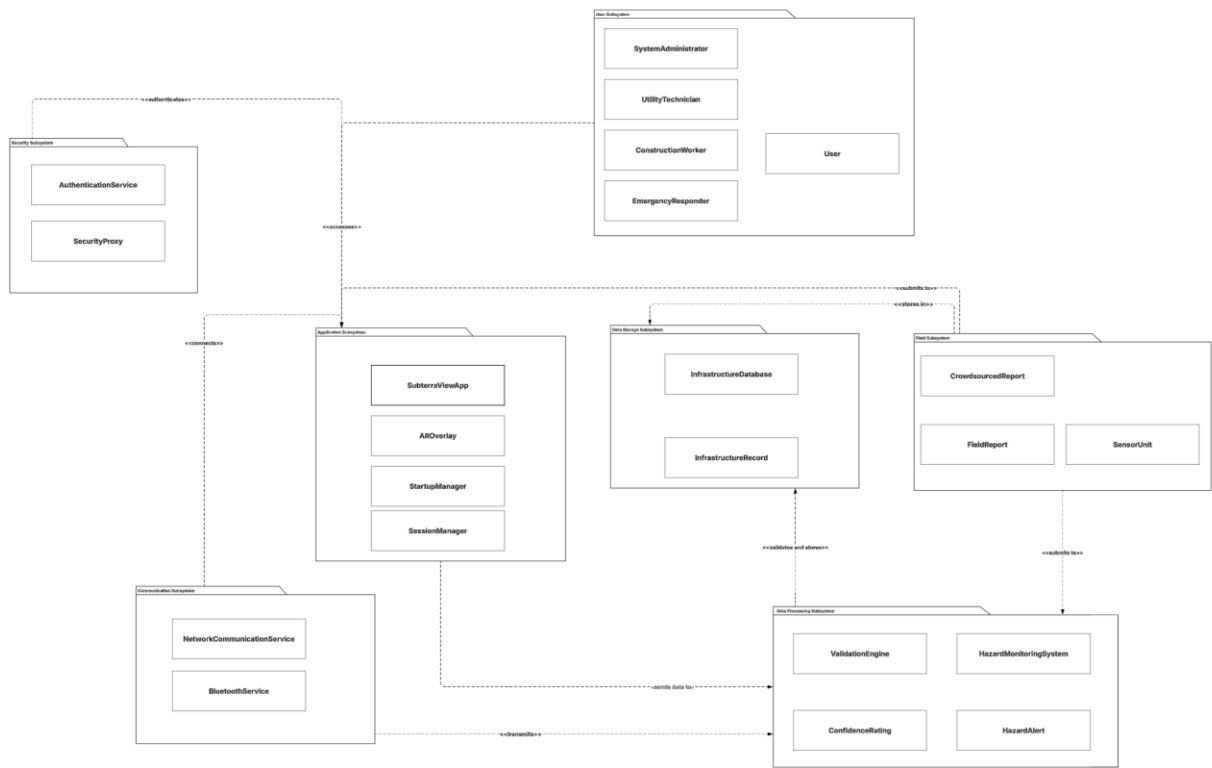


Figure 12 - Final Package Diagram

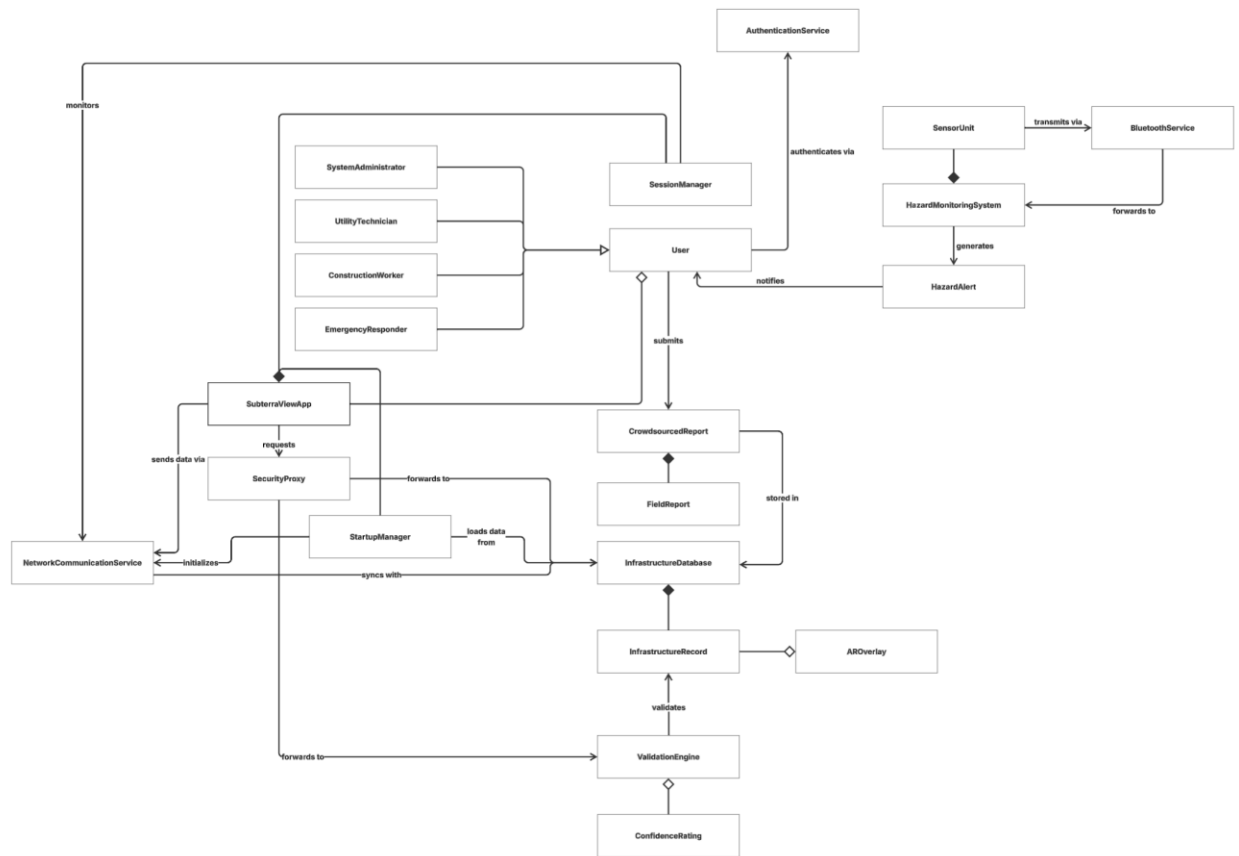


Figure 13 - Final Class Diagram

The final system design incorporates all classes and subsystems identified throughout Sections 24 and 25. The class diagram above presents the complete set of 22 classes and their relationships, while the package diagram shows the same classes partitioned into seven subsystems. This expands on the initial decomposition from Section 24d with the addition of a Security Subsystem and a Communication Subsystem identified as a result of the additional design considerations in Section 25.

The Security Subsystem was added to handle access control and authentication concerns identified in Section 25c. It contains the AuthenticationService class, which verifies user credentials and assigns access levels at login, and the SecurityProxy class, which filters all requests from the SubterraViewApp before they reach the Data Processing or Data Storage subsystems.

The Communication Subsystem was added because of the hardware mapping in Section 25a. It contains the NetworkCommunicationService class, which manages all HTTPS communication between the User Smartphone and the Cloud Server, and the BluetoothService class, which handles real time sensor data transmission between the Sensor Unit and the User Smartphone.

The Application Subsystem was updated to include the SessionManager class identified in Section 25d and the StartupManager class identified in Section 25e.

These classes handle user session management and system initialization respectively, ensuring the app starts up and shuts down reliably across all supported devices.

27 Object Design

27a Packages

SubterraView's classes are organized into seven packages corresponding to the subsystems identified in Section 26. Each package groups classes that are semantically related and work closely together, following the principle of maximizing cohesion within packages and minimizing coupling between them.

The `com.subterraview.user` package contains all user role classes including `User`, `ConstructionWorker`, `UtilityTechnician`, `EmergencyResponder`, and `SystemAdministrator`. The `com.subterraview.app` package contains the core application classes `SubterraViewApp`, `AROverlay`, `SessionManager`, and `StartupManager`. The `com.subterraview.processing` package contains the data processing classes `ValidationEngine`, `ConfidenceRating`, `HazardMonitoringSystem`, and `HazardAlert`. The `com.subterraview.field` package contains the field data collection classes `SensorUnit`, `FieldReport`, and `CrowdsourcedReport`. The `com.subterraview.storage` package contains the data storage classes `InfrastructureDatabase` and `InfrastructureRecord`. The `com.subterraview.security` package contains the security classes `AuthenticationService` and `SecurityProxy`. The `com.subterraview.communication` package contains the communication classes `NetworkCommunicationService` and `BluetoothService`.

27b User Subsystem

The User Subsystem defines the roles and access levels of all human actors that interact with SubterraView. The base `User` class captures attributes and behaviors shared across all user types, while the four subclasses specialize behavior based on role. A key constraint across all user classes is that `accessLevel` must be assigned at login and cannot be changed during an active session.

User: The base class for all authenticated actors in the system. Responsible for managing login state and providing role and access level information to other parts of the system.

Constraint(s): Role must be one of the four defined user types and `accessLevel` must be a non-negative integer.

```
public class User {  
  
    private String userID;  
  
    private String username;  
  
    private String password;  
  
    private String role;
```

```

private int accessLevel;

public void login(String username, String password)

public void logout()

}

```

ConstructionWorker: Responsible for viewing underground infrastructure and acknowledging hazard alerts in the field.

Constraint(s): acknowledgeHazard() requires a valid alertID that exists in the system.

```

public class ConstructionWorker extends User {

    public void viewAROverlay()

    public void acknowledgeHazard(String alertID)

}

```

UtilityTechnician: A technician who verifies infrastructure records and submits crowdsourced field reports. Responsible for verifying infrastructure records and submitting crowdsourced field reports.

Constraint(s): submitFieldReport() requires location and infrastructureType to be non-null.

```

public class UtilityTechnician extends User {

    public void submitFieldReport(String location, String type, String description,
String images)

    public void flagDiscrepancy(String recordID)

}

```

EmergencyResponder: A first responder who accesses SubterraView during active incidents. Responsible for accessing emergency mode and priority infrastructure maps during active incidents.

Constraint(s): activateEmergencyMode() requires a valid location parameter.

```

public class EmergencyResponder extends User {

    public void activateEmergencyMode(String location)

```

```

    public void viewPriorityMap(String location)
}

```

SystemAdministrator: Responsible for managing data integrity and system maintenance. Has full access to all system functions.

Constraint(s): approveRecord() and rejectRecord() require a valid recordID that exists in the system.

```

public class SystemAdministrator extends User {

    public void reviewDataAccuracyReport()

    public void approveRecord(String recordID)

    public void rejectRecord(String recordID)

}

```

27c Application Subsystem

The Application Subsystem serves as the primary interface between the user and the rest of the system. It handles all user interactions, displays infrastructure visualizations, delivers hazard alerts, and manages the application lifecycle across iOS and Android platforms.

SubterraViewApp: The core mobile application class that manages all user interactions and coordinates communication between the UI layer and the backend subsystems.

Constraint(s): currentUser must be authenticated before any core features can be accessed.

```

public class SubterraViewApp {

    private User currentUser;

    private SessionManager sessionManager;

    private StartupManager startupManager;

    private boolean isOnline;

    public void openApp()

    public void closeApp()
}

```

```

public void confirmLocation(String gps)

public void requestInfrastructureData(String location)

public void displayAROverlay(AROverlay overlay)

public void displayHazardAlert(HazardAlert alert)

public void displayEmergencyMap(String location)

public void submitReport(FieldReport report)

public void contact811()

public void switchScreen(String screenName)

}

```

AROverlay: Responsible for building and rendering the color coded augmented reality visualization of underground infrastructure on the user's device screen.

Constraint(s): buildOverlay() requires a non-empty list of InfrastructureRecord objects with valid location data.

```

public class AROverlay {

    private List<InfrastructureRecord> records;

    private double confidenceThreshold;

    private String overlayMode;


    public void buildOverlay(List<InfrastructureRecord> records)

    public void renderOverlay()

    public void updateOverlay(InfrastructureRecord record)

    public void highlightHazardZone(String location)

    public void clearOverlay()

}

```

SessionManager: Responsible for managing the active user session on the device. Implemented using the Singleton pattern, meaning only one instance of

SessionManager can exist at any time. The getInstance() method is documented here because it is the controlled access point required by the Singleton pattern.

Constraint(s): only one active session can exist per device at any time.

```
public class SessionManager {  
    private static SessionManager instance;  
  
    private User activeUser;  
  
    private boolean isOnline;  
  
    private String sessionToken;  
  
  
    public static SessionManager getInstance() // Singleton access point  
  
    public void startSession(User user)  
  
    public void endSession()  
  
    public void checkConnectivity()  
  
    public void handleOfflineMode()  
  
    public boolean isSessionActive()  
  
}
```

StartupManager: Responsible for handling system initialization when the app launches, ensuring all required components are online before the user can interact with the system.

Constraint(s): initializeApp() must complete successfully before any other system function is accessible.

```
public class StartupManager {  
  
    private boolean sensorsActive;  
  
    private boolean networkAvailable;  
  
    private boolean cacheLoaded;  
  
  
    public void initializeApp()  
  
    public void loadCachedData()
```

```

    public void checkSensorConnection()

    public void verifyNetworkConnection()

    public void handleAbnormalShutdown()

}

```

27d Data Processing Subsystem

The Data Processing Subsystem is responsible for all data validation, hazard detection, and alert generation within the system. It contains the core AI logic that drives SubterraView's infrastructure mapping and safety monitoring capabilities.

ValidationEngine: The AI component responsible for cross-referencing infrastructure records from multiple sources, assigning confidence ratings, and flagging discrepancies for review. This is the most critical class in the system as all infrastructure data passes through it before being displayed to users.

Constraint(s): confidenceThreshold must be between 0.0 and 1.0, and all incoming records must be assigned a confidence rating before integration.

```

public class ValidationEngine {

    private double confidenceThreshold;

    private List<String> dataSources;

    private InfrastructureDatabase database;


    public void validateRecord(InfrastructureRecord record)

    public ConfidenceRating assignConfidenceRating(InfrastructureRecord record)

    public void crossReferenceData(List<InfrastructureRecord> records)

    public void flagForReview(InfrastructureRecord record)

    public void storeApprovedRecord(InfrastructureRecord record)

    public void checkThreshold(InfrastructureRecord record)

}

```

ConfidenceRating: Represents the reliability score assigned to an infrastructure record by the ValidationEngine. Stores the score and the threshold used to determine whether a record is approved or flagged for review.

Constraint(s): score must be between 0.0 and 1.0 and threshold must match the system defined minimum confidence level.

```
public class ConfidenceRating {  
  
    private double score;  
  
    private double threshold;  
  
    private String status;  
  
    public boolean meetsThreshold()  
  
}
```

HazardMonitoringSystem: Receives real time sensor readings from deployed SensorUnit devices and determines when a hazard condition has been met. Implements the Observer pattern to notify multiple components simultaneously when a hazard is detected.

Constraint(s): alertThreshold must be a positive value and at least one observer must be registered before alerts can be dispatched.

```
public class HazardMonitoringSystem {  
  
    private List<SensorUnit> activeSensors;  
  
    private double alertThreshold;  
  
    private List<Observer> observers;  
  
    public void receiveReading(SensorUnit sensor, double reading)  
  
    public void analyzeReading(double reading)  
  
    public HazardAlert generateAlert(String location, String hazardType)  
  
    public void registerObserver(Observer observer)  
  
    public void removeObserver(Observer observer)  
  
    public void notifyObservers(HazardAlert alert)  
  
}
```

HazardAlert: Represents a hazard event generated by the HazardMonitoringSystem. Stores all relevant details about the hazard and is responsible for notifying relevant users through the application.

Constraint(s): hazardType and severity must be non-null, and timestamp must be set at the time of creation.

```
public class HazardAlert {  
  
    private String alertID;  
  
    private String hazardType;  
  
    private String severity;  
  
    private String location;  
  
    private DateTime timestamp;  
  
    private boolean acknowledged;  
  
    public void sendNotification(User user)  
  
    public void acknowledge(String userID)  
  
}
```

27e Field Subsystem

The Field Subsystem is responsible for collecting data from the physical environment. It manages incoming sensor readings from deployed hardware units and handles the submission and storage of crowdsourced field reports submitted by users in the field.

SensorUnit: Represents an individual external Bluetooth sensor deployed at a job site. Detects underground hazards such as gas leaks and structural instabilities and transmits real time readings to the HazardMonitoringSystem through the BluetoothService.

Constraint(s): transmitReading() can only be called when isActive is true.

```
public class SensorUnit {  
  
    private String sensorID;  
  
    private String location;  
  
    private boolean isActive;  
  
    private double currentReading;
```

```

    public void activate()

    public void deactivate()

    public void transmitReading(HazardMonitoringSystem system)

}

```

FieldReport: Represents a crowdsourced infrastructure report submitted by a user in the field. Stores the infrastructure location, type, description, and any attached images.

Constraint(s): location and infrastructureType must be non-null before submitReport() can be called.

```

public class FieldReport {

    private String reportID;

    private String location;

    private String infrastructureType;

    private String description;

    private List<String> images;

    private DateTime submissionTime;


    public void submitReport()

}

```

CrowdsourcedReport: Manages the storage and review status of submitted field reports. Forwards reports to the ValidationEngine for processing and stores approved reports in the InfrastructureDatabase.

Constraint(s): validateReport() must be called before storeReport() to ensure no unvalidated reports are stored.

```

public class CrowdsourcedReport {

    private String reportID;

    private FieldReport fieldReport;

```

```

private String status;

private DateTime reviewTime;


public void createReport(FieldReport report)

public void validateReport(ValidationEngine engine)

public void storeReport(InfrastructureDatabase database)

public void updateStatus(String status)

}

```

27f Data Storage Subsystem

The Data Storage Subsystem is responsible for storing and managing all validated infrastructure records within the system. It serves as the central data layer that all other subsystems depend on to retrieve and persist infrastructure data.

InfrastructureDatabase: The primary storage class for all validated infrastructure records, crowdsourced reports, and user account data. All persistent data in SubterraView flows through this class.

Constraint(s): connect() must be called before any read or write operations and no records should be written during an active transaction failure.

```

public class InfrastructureDatabase {

    private String databaseID;

    private List<InfrastructureRecord> records;

    private List<CrowdsourcedReport> reports;

    private boolean isConnected;


    public void storeRecord(InfrastructureRecord record)

    public void deleteRecord(String recordID)

    public InfrastructureRecord getRecord(String recordID)

    public List<InfrastructureRecord> queryRecords(String location)

    public void storeReport(CrowdsourcedReport report)

    public void connect()

```

```

    public void disconnect()
}

```

InfrastructureRecord: Represents a single underground infrastructure element stored in the system. Stores all relevant attributes of the infrastructure including type, location, depth, and confidence rating.

Constraint(s): infrastructureType must be one of the defined utility types and depth must be a positive value.

```

public class InfrastructureRecord {

    private String infrastructureID;

    private String infrastructureType;

    private String location;

    private double depth;

    private ConfidenceRating confidenceRating;

    private DateTime lastUpdated;

    public void updateRecord(String type, String location, double depth)

    public boolean isVerified()

}

```

27g Security Subsystem

The Security Subsystem is responsible for managing access control and protecting sensitive infrastructure and user data throughout the system. It ensures that users can only access the features and data appropriate for their assigned role.

AuthenticationService: Responsible for verifying user credentials at login, assigning the correct access level based on the user role, and enforcing access restrictions throughout the session. Implemented using the Singleton pattern to ensure only one authentication service exists at a time. The getInstance() method is documented here because it is the controlled access point required by the Singleton pattern.

Constraint(s): only one instance can exist at a time and encryptionKey must be set before verifyCredentials() can be called.

```

public class AuthenticationService {

```

```

private static AuthenticationService instance;

private List<User> registeredUsers;

private String encryptionKey;


public static AuthenticationService getInstance()

public boolean verifyCredentials(String username, String password)

public int assignAccessLevel(String role)

public void enforceAccess(User user, String feature)

public boolean isAuthenticated(String userID)

public void logout(String userID)

}

```

SecurityProxy: Sits between the SubterraViewApp and the backend services, filtering all incoming requests based on the authenticated user role before they reach the Data Processing or Data Storage subsystems. Implements the Proxy pattern as identified in Section 25g.

Constraint(s): authenticatedUser must be set before filterRequest() can be called.

```

public class SecurityProxy {

    private User authenticatedUser;

    private AuthenticationService authService;

    private List<String> allowedFeatures;


    public void filterRequest(String request, User user)

    public boolean hasAccess(User user, String feature)

    public void forwardRequest(String request)

    public void denyRequest(String request)

    public void logAccessAttempt(User user, String feature)

}

```

27h Communication Subsystem

The Communication Subsystem is responsible for managing all data transfer between the hardware nodes identified in Section 25a. It handles both the HTTPS communication between the User Smartphone and the Cloud Server and the Bluetooth communication between the Sensor Unit and the User Smartphone.

NetworkCommunicationService: Manages all HTTPS communication between the User Smartphone and the Cloud Server. Handles sending user requests, receiving validated infrastructure data, and delivering hazard alerts back to the app. Also manages local caching of infrastructure data in low connectivity environments.

Constraint(s): serverURL must be set before sendRequest() can be called and cachedRecords must not exceed the device storage limit.

```
public class NetworkCommunicationService {  
  
    private String serverURL;  
  
    private boolean isConnected;  
  
    private List<InfrastructureRecord> cachedRecords;  
  
    private SessionManager sessionManager;  
  
    public void sendRequest(String request)  
  
    public void receiveResponse(String response)  
  
    public void cacheData(List<InfrastructureRecord> records)  
  
    public List<InfrastructureRecord> loadCachedData()  
  
    public void syncWithServer()  
  
    public void handleConnectionLoss()  
  
}
```

BluetoothService: Handles the real time Bluetooth communication between the Sensor Unit and the User Smartphone. Manages sensor pairing, connection monitoring, and transmission of sensor readings to the HazardMonitoringSystem.

Constraint(s): forwardReading() can only be called after a sensor has been successfully paired.

```
public class BluetoothService {
```

```

private List<SensorUnit> pairedSensors;

private boolean isBluetoothActive;

private HazardMonitoringSystem hazardSystem;

public void pairSensor(SensorUnit sensor)

public void unpairSensor(SensorUnit sensor)

public void receiveReading(SensorUnit sensor, double reading)

public void forwardReading(double reading, HazardMonitoringSystem system)

public void handleConnectionLoss(SensorUnit sensor)

}

```

IV Project Issues

28 Open Issues

28a Access to Accurate Utility Data

It is currently unknown whether all utility companies/municipal agencies will provide consistent, real-time access to underground utility data. Restrictions, data privacy concerns, and outdated records limit accuracy.

28b Data Standardization Across Sources

Different entities store infrastructure data in varying formats. A unified standard for integrating these datasets has not yet been finalized, which may affect the AI validation process, as a custom method would have to be used to access data in each unique data format.

28c AR Inaccuracies

The effectiveness of AR visualization in different field conditions (lighting, weather, obstructions) has not yet been fully tested.

28d 811 Integration

Integration with 811 services is not fully defined, particularly how real-time data synchronization will occur.

28e Legal Liability

Potential regulatory or legal changes regarding underground data sharing and liability are unknown currently.

28f Crowdsourcing Validation

Potential regulatory or legal changes regarding underground data sharing and liability are unknown currently.

29 Off-the-Shelf Solutions

There are multiple OtS Solutions that can be used within SubterraView. Their main use cases are to minimize the number of custom software and hardware the developers would have to make.

29a Ready-Made Products

GIS platforms(i.e. ESRI ArcGIS) for managing and visualizing geospatial data as well as infrastructure data

Ground-Penetrating Radar systems for underground detection

8-1-1 services platforms

AR platforms such as ARKit, ARCore

29b Reusable Components

Commercial gas detection devices

Machine learning frameworks such as TensorFlow or PyTorch

Mapping libraries such as Mapbox or Google Maps SDK

Cloud services (AWS, Azure, Google Cloud) for data storage and Processing

Mobile development frameworks for iOS and Android

Database systems for managing infrastructure and user-generated data, such as Supabase or Firebase

29c Products That Can Be Copied

Existing GIS-based infrastructure mapping systems can serve as a structural model for data organization

Prior academic or industry projects involving AR visualization can inform UI/UX design

Existing crowdsourcing platforms (e.g., reporting systems) can guide user submission workflows

30 New Problems

30a Effects on the Current Environment

Construction workflows may shift significantly, reducing reliance on traditional 811 marking processes

Workers may require training to effectively use AR-based tools which would require man-hours and funding

Over-reliance on the system could reduce manual verification practices and in turn cause issues if systems go down

30b Effects on the Installed Systems

Integration with existing utility databases and city systems may introduce compatibility challenges

Data format inconsistencies across organizations will require transformations

Existing systems may not support real-time updates required by SubterraView

30c Potential User Problems

Users may distrust AI-generated infrastructure data, especially in high-risk environments

Learning curve for AR interfaces could slow adoption among less tech-savvy workers

Existing crowdsourcing platforms (e.g., reporting systems) can guide user submission workflows

30d Limitations in the Anticipated Implementation Environment That May Inhibit the New Product

Poor internet connectivity at construction sites may limit real-time data access

Weather conditions (rain, fog, dust) may affect AR visibility and sensor accuracy

Battery limitations on mobile devices and sensors may impact continuous usage

30e Follow-Up Problems

High user demand could strain cloud infrastructure and system performance

Legal liability issues if inaccurate data leads to damages or injuries

Scaling the system across multiple cities with different standards and data formats

31 Migration to the New Product

N/A – SubterraView does not replace any single system.

32 Risks

Risk	Probability	Impact
Inaccurate or incomplete infrastructure data	High	High
Integration failures with external systems	Medium	High

AR performance issues in real-world environments	Medium	Medium
Sensor hardware malfunction or inaccuracy	Medium	High
User resistance to adopting new technology	Medium	Medium
Legal/regulatory challenges	Low	High

Table 6 - Risks

33 Costs

33a Development Costs

Software engineering, AI model development, AR implementation (~\$80,000–\$150,000)

33b Hardware Costs

Sensor units and testing equipment (~\$20,000–\$50,000)

33c Testing & Development

Field testing, bug fixes, and user training (~\$20,000–\$40,000)

33d Total Estimated Initial Monetary Cost

\$130,000 – \$270,000 (initial development phase)

33e Data Acquisition

Potential fees for access utility databases

33f Maintenance & Updates

Recurring costs for system updates, bug fixes, and AI model training

33g Scalability

Increased cloud and infrastructure expenses as user base grows

33h Total Man-Hours

35-40 hours a week per member, 4-6 members ~6,000-10,000 total man-hours

33i Development Time

12 months development, 6 months testing

33j Ongoing Annual Costs, Post-Deployment

\$20,000-\$50,000

34 Waiting Room

Integration with wearable AR devices (e.g., smart glasses)

Predictive analytics for infrastructure failure detection

Offline mode for low-connectivity environments

3D underground simulation and playback features

Integration with autonomous construction equipment

35 Ideas for Solutions

Use ARCore/ARKit for mobile AR implementation

Implement a confidence scoring system using machine learning models

Use a microservices architecture for scalability

36 Project Retrospective

One of the most successful aspects of this project was communication within our team. We consistently stayed in touch through messaging platforms and regular check-ins, which allowed us to coordinate progress effectively and avoid major misunderstandings. This made it easier to divide tasks and ensure that everyone was aligned on project goals and deadlines. Reaching out and talking to professionals in this field also allowed us to make ideas that would be useful to real workers.

What didn't work as well was online meetings that were set up. Since these tended to run longer and with a less clear goal, team members tended to get sidetracked on goals. In the future, running with clear goals for the team will make it easier to ensure more productive sessions.

Overall, as this project continues, to ensure better efficiency and productivity, I recommend divvying tasks up, much like how developers use story points. Splitting the group into twos and working on sections of the project together to allow for parts of the project to advance in parallel. This story point XP style of task distribution will allow for more efficient workloads.

V Glossary

811 — The national call before you dig service in the United States that notifies utility companies of planned excavations so they can mark underground lines before digging begins. SubterraView is designed to supplement this service, not replace it.

Access Level — A non-negative integer assigned to each user role that determines which features and data a user can access within SubterraView.

Artificial Intelligence (AI) — A computational system that takes large datasets, detects patterns, and makes decisions or predictions based on the information given. In SubterraView, AI is used to cross-reference underground infrastructure data from multiple sources, assign confidence ratings, and update the infrastructure model.

AR Overlay — The color coded visual layer rendered on top of the device camera feed showing the location and type of underground infrastructure beneath the surface.

ARCore — Google's augmented reality SDK for Android devices, used in SubterraView to power the AR visualization on Android platforms.

ARKit — Apple's augmented reality SDK for iOS devices, used in SubterraView to power the AR visualization on iOS platforms.

API — Application Programming Interface. A defined interface that allows different software systems to communicate and exchange data.

Augmented Reality (AR) — A technology that overlays digital information and visual elements onto the real world environment using devices such as smartphones. In SubterraView, augmented reality is used to display underground infrastructure directly over a live camera view.

AuthenticationService — A class in the Security Subsystem responsible for verifying user credentials at login, assigning the correct access level based on the user role, and enforcing access restrictions throughout the session. Implemented using the Singleton pattern.

AWS — Amazon Web Services. A cloud computing platform used to host SubterraView's backend services, AI validation engine, and data storage.

BluetoothService — A class in the Communication Subsystem responsible for managing Bluetooth communication between deployed SensorUnit devices and the User Smartphone.

Boundary Condition — A situation outside of normal steady state system operation, including startup, shutdown, and exception handling scenarios.

CrowdsourcedReport — A class in the Field Subsystem that manages the storage and review status of field reports submitted by users. Forwards reports to the ValidationEngine for processing.

Crowdsourcing — The process of collecting information from field workers or system users who submit reports about infrastructure discrepancies, new utilities, or hazard observations through the SubterraView application.

Confidence Rating — A numerical score between 0.0 and 1.0 generated by the ValidationEngine that represents the reliability and verification level of a mapped infrastructure record.

Client-Server — A software architecture pattern where servers provide services independently of clients and clients may contact multiple servers. SubterraView uses this pattern for communication between the User Smartphone and the Cloud Server, and between the Sensor Unit and the smartphone.

ConfidenceRating — A class in the Data Processing Subsystem that stores the reliability score assigned to an infrastructure record by the ValidationEngine, along with the threshold used to determine whether the record is approved or flagged for review.

Deployment Diagram — A UML diagram that shows how software subsystems are mapped across physical hardware nodes and what communication protocols connect them.

Event-Driven Control — A software control approach where the system responds to events as they occur rather than following a fixed sequence of procedures. SubterraView uses event-driven control to handle hazard alerts and real time data updates.

FieldReport — A class in the Field Subsystem representing a crowdsourced infrastructure report submitted by a user in the field, including location, infrastructure type, description, and attached images.

GIS — Geographic Information System. A framework for capturing, storing, analyzing, and displaying geospatial data.

GLTF — GL Transmission Format. An open standard file format for three-dimensional scene and model data used to store AR overlay data in SubterraView.

GPS — Global Positioning System. A satellite-based navigation system used to determine precise geographic coordinates of a user's location in SubterraView.

GPR — Ground Penetrating Radar. A geophysical method that uses radar pulses to detect subsurface objects and structures.

HazardAlert — A class in the Data Processing Subsystem representing a hazard event generated by the HazardMonitoringSystem. Stores hazard type, severity, location, and timestamp and is responsible for notifying relevant users.

HazardMonitoringSystem — A class in the Data Processing Subsystem that receives real time sensor readings and determines when a hazard condition has been met. Implements the Observer pattern to notify multiple components simultaneously when a hazard is detected.

Hazard Monitoring System — A system module that receives input from sensor units and analyzes environmental risks to provide real time safety alerts to workers and emergency responders.

InfrastructureDatabase — A class in the Data Storage Subsystem that serves as the primary storage class for all validated infrastructure records, crowdsourced reports, and user account data.

InfrastructureRecord — A class in the Data Storage Subsystem representing a single underground infrastructure element, storing utility type, location coordinates, depth, and confidence rating.

iOS — iPhone Operating System. Apple's mobile operating system used on iPhones and iPads. One of the two platforms SubterraView is designed to run on.

JDBC — Java Database Connectivity. A standard API used to connect the Cloud Server to the Database Server in SubterraView's deployment architecture.

Model-View-Controller (MVC) — A software architecture pattern that separates a system into three components: the Model which holds the data, the View which displays it, and the Controller which processes input and updates the model. SubterraView uses MVC to keep data, logic, and interface cleanly separated.

NetworkCommunicationService — A class in the Communication Subsystem responsible for managing all HTTPS communication between the User Smartphone and the Cloud Server, including local caching of infrastructure data in low connectivity environments.

Observer Pattern — A behavioral design pattern where an object called the subject maintains a list of observers and notifies them automatically when an event occurs. Applied in SubterraView to the HazardMonitoringSystem to notify multiple components simultaneously when a hazard is detected.

OSHA — Occupational Safety and Health Administration. A federal agency that sets and enforces workplace safety standards relevant to SubterraView's hazard alert requirements.

Package Diagram — A UML diagram that groups classes into subsystems or packages, showing the dependencies between them. Used in SubterraView to illustrate the seven subsystem decomposition.

PostGIS — A geospatial extension for PostgreSQL that adds support for geographic data types. Used in SubterraView to store and query location based infrastructure data.

PostgreSQL — A relational database system used in SubterraView to store and manage all persistent infrastructure records, crowdsourced reports, and user account data.

Proxy Pattern — A structural design pattern where a proxy object sits between a client and a real subject, controlling access. Applied in SubterraView through the SecurityProxy class to filter requests based on user role before they reach backend services.

Role-Based Access Control (RBAC) — A security approach where users are granted access to features and data based on their assigned role. SubterraView uses RBAC to ensure each user type only accesses what their role permits.

SDK — Software Development Kit. A set of tools, libraries, and documentation used to build applications for a specific platform.

SecurityProxy — A class in the Security Subsystem that sits between the SubterraViewApp and backend services, filtering all incoming requests based on the authenticated user role. Implements the Proxy pattern.

SensorUnit — A class in the Field Subsystem representing an individual external Bluetooth sensor deployed at a job site that detects underground hazards and transmits real time readings to the HazardMonitoringSystem.

Sensor Units — Hardware devices that detect underground hazards such as gas leaks, structural instability, or abnormal environmental conditions and transmit alerts to the SubterraView system.

SessionManager — A class in the Application Subsystem responsible for managing the active user session on the device. Implements the Singleton pattern to ensure only one session exists at a time.

Singleton Pattern — A creational design pattern that ensures only one instance of a class can exist at any time and provides a global access point to that instance. Applied in SubterraView to the SessionManager and AuthenticationService classes.

StartupManager — A class in the Application Subsystem responsible for handling system initialization when the app launches, ensuring all required components are online before the user can interact with the system.

SubterraViewApp — The core mobile application class in the Application Subsystem that manages all user interactions and coordinates communication between the UI layer and the backend subsystems.

Subterranean Infrastructure — Underground systems that support municipal and utility operations, including water pipelines, sewage systems, gas lines, electrical conduits, and telecommunications cables.

TensorFlow / PyTorch — Machine learning frameworks used to build and deploy the AI validation engine responsible for cross-referencing infrastructure data and assigning confidence ratings.

ValidationEngine — A class in the Data Processing Subsystem representing the AI component responsible for cross-referencing infrastructure records from multiple sources, assigning confidence ratings, and flagging discrepancies for review.

VI References / Bibliography

- [1] J. Bell, "Underwater Archaeological Survey Report Template: A Sample Document for Generating Consistent Professional Reports," Underwater Archaeological Society of Chicago, Chicago, 2012.
- [2] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Ninth ed., Wiley, 2013.
- [3] Robertson and Robertson, Mastering the Requirements Process.
- [4] M. Fowler, UML Distilled, Third Edition, Boston: Pearson Education, 2004.

- [5] Common Ground Alliance, "2024 DIRT Report: Damage Information Reporting Tool," Washington, DC, 2025.
- [6] Associated General Contractors of America, "Most Contractors Report Significant Flaws With The Nation's 811 Utility Location System, Citing Inaccurate Locates And Slow Response Times," Associated General Contractors of America, Nov 2021. [Online]. Available: <https://www.agc.org/news/2021/11/17/most-contractors-report-significant-flaws-nation%E2%80%99s-811-utility-location-system>. [Accessed 2026].
- [7] American Public Works Association, "About Public Works," American Public Works Association, [Online]. Available: <https://www.apwa.org/resources/about-public-works/>. [Accessed 2026].
- [8] Mordor Intelligence, "Underground Utility Mapping Market Size & Share Analysis," Mordor Intelligence, [Online]. Available: <https://www.mordorintelligence.com/industry-reports/underground-utility-mapping-market>. [Accessed 2026].

VII Index

811.....	24	MVC	69, 97
AROverlay	23, 69, 78, 81	NetworkCommunicationService..	24, 71, 72, 73, 75, 77, 78, 90, 98
Augmented Reality	8, 20, 96	Observer Pattern.....	98
AuthenticationService	24, 72, 77, 78, 88, 89, 96, 99	Proxy pattern	75, 89, 98
BluetoothService.....	71, 77, 78, 85, 90, 96	Requirements	43
Client-Server	69, 96	SecurityProxy.....	72, 75, 77, 78, 89, 98
confidence rating..	13, 17, 18, 31, 32, 35, 36, 41, 43, 44, 45, 60, 74, 83, 88, 97	SensorUnit.....	23, 78, 84, 85, 91, 96, 98
ConfidenceRating	72, 78, 83, 84, 88, 96	SessionManager ...	73, 75, 77, 78, 80, 81, 82, 90, 99
CrowdsourcedReport	78, 86, 87, 96	Singleton Pattern	99
Crowdsourcing.....	21, 91, 96	SubterraView .	1, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 24, 25, 29, 30, 31, 32, 33, 34, 35, 37, 38, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 54, 55, 56, 57, 63, 64, 65, 69, 70, 71, 72, 73, 78, 79, 83, 87, 93, 95, 96, 97, 98, 99
Design	63	SubterraViewApp	22, 23, 24, 69, 72, 75, 77, 78, 80, 89, 98, 99
FieldReport	78, 81, 86, 87, 97	ValidationEngine .	22, 23, 24, 69, 71, 78, 83, 86, 87, 96, 99
HazardAlert.....	75, 78, 81, 84, 85, 97		
HazardMonitoringSystem	23, 69, 71, 73, 75, 78, 84, 85, 86, 90, 91, 97, 98		
InfrastructureDatabase	22, 23, 69, 72, 78, 83, 86, 87, 97		
InfrastructureRecord ..	69, 72, 78, 81, 83, 87, 88, 90, 97		
Model-View-Controller	69, 97		